EQuADiSE: Efficient Quantum-safe Adaptive Distributed Symmetric-key Encryption

Sayani Sinha[1], Sikhar Patranabis[2], and Debdeep Mukhopadhyay[1]

[1]IIT Kharagpur India
[2]IBM Research India

April 22, 2026

Abstract

Distributed symmetric-key encryption (DiSE), introduced in CCS'18 enables threshold versions of traditional (symmetric-key) authenticated encryption. In DiSE, the long-term master secret key is secret-shared among multiple parties following a threshold access structure, and both encryption and decryption are performed in a distributed manner. An adaptively secure DiSE, introduced in INDOCRYPT '20 tolerates adaptive corruptions of the key-holding parties for arbitrary thresholds, while simultaneously retaining efficient encryption and decryption. Unfortunately, all existing instances of adaptively secure DiSE are either quantum-broken (due to their inherent-reliance on discrete log-hard groups), or incur exponential (in the number of parties) online overheads for encryption/decryption.

In this paper, we present EQuADiSE – the first practically efficient, adaptively secure, and plausibly post-quantum construction of DiSE based on the Module Learning with Rounding (MLWR) assumption in the Quantum Random Oracle model (QROM). EQuADiSE is the first adaptively secure quantum-safe instance of DiSE that incurs linear (in the number of parties) encryption/decryption overheads. As a core technical tool of independent interest, we introduce an MLWR-based distributed pseudorandom function (DPRF) that enjoys adaptive security in the QROM and practically outperforms *all* existing adaptively secure DPRF constructions in terms of online evaluation time.

We present experimental evaluations demonstrating that EQuADiSE achieves higher online throughput than *all* prior realizations of DiSE, including quantum-broken realizations based on discrete log-hard groups.

Contents

1 Introduction

Threshold cryptography [20] enables protecting long-term secret keys by secret-sharing such keys among multiple parties. Any attacker corrupting upto a threshold number of these parties (information-theoretically) fails to recover the secret keys. Threshold cryptography serves as the foundation for several advanced cryptographic primitives such as threshold fully-homomorphic encryption [10], secure multiparty computation [5], function secret sharing [13], and functional encryption [25]. Traditionally, threshold cryptography considers two kinds of corruption models – *static*, where the adversary corrupts a fixed subset of the key-holding parties at the beginning of time, and *adaptive*, where the adversary adaptively chooses which key-holding parties to corrupt over time. For many practical applications, security against adaptive corruptions is more desirable.

Distributed Symmetric-Key Encryption. The study of threshold authenticated symmetric-key encryption was introduced in [2]. The authors of [2] introduced a primitive called distributed symmetric-key encryption (DiSE). In DiSE, the master secret key is secret-shared among multiple parties following a threshold access structure, and both encryption and decryption are performed in a distributed manner. DiSE enables protecting long-term keys in many different applications, such as protecting data at rest, authenticating clients on enterprise networks, and securing data and payments on IoT devices.

Informally speaking, the DiSE scheme introduced in [2] (followed by several other optimizations of DiSE [17, 1, 21, 28]) uses the following blueprint. Consider the classic construction of symmetric-key encryption (SKE) from a pseudorandom function (PRF) where the encryption of a message m under a secret key k is given by

$$\mathsf{Enc}(k, m) = (r, F_k(r) \oplus m),$$

where F_k is a PRF and r is a randomly sampled PRF input. The core idea of [2] is to distribute the encryption/decryption procedure by replacing the PRF F_k by a distributed PRF (DPRF) [30]. A DPRF allows securely evaluating $F_k(\cdot)$ on any input in a distributed manner, while the secret key k is secret-shared across multiple parties following a threshold access structure. There exist known constructions of DPRFs from purely symmetric-key assumptions such as any PRF [30], as well constructions from number-theoretic assumptions such as the decisional Diffie-Hellman (DDH) assumption [30] and lattice-based assumptions such as Learning with Errors (LWE) [24], Learning with Rounding (LWR) [35]. Following the blueprint of [2], these instantiations of DPRF yields DiSE schemes with varying efficiency properties.

A DiSE scheme is said to be *ideally efficient* if: (i) the key storage required by each party is linear in the number of key-holding parties, and (ii) the online evaluation time for encryption/decryption is linear in the number of key-holding parties. In practice, the second requirement is often more important, particularly for applications (such as authenticating clients on enterprise networks) where ensuring fast online turnaround time is crucial.

Adaptive Security. A DiSE scheme is said to be *adaptively* secure if it tolerates adaptive corruptions of the key-holding parties for arbitrary thresholds, while simultaneously retaining efficient encryption and decryption. Among the known instantiations of DiSE, the one based on purely symmetric-key realizations of DPRF is adaptively secure, but is asymptotically inefficient since it incurs *both* exponential key storage *and* exponential online overheads for encryption/decryption. The only efficient yet adaptively secure DiSE scheme is based on an

adaptive DPRF from the DDH assumption [29]. While this construction is ideally efficient and plausibly secure against classical attackers, it is broken by known quantum algorithms for discrete log [34].

Post-Quantum Landscape for Adaptive DiSE. In this paper, we are interested in building practically efficient, adaptively secure, and plausibly post-quantum instances of DiSE. Note that such a construction follows immediately from a DPRF satisfying the same set of properties simultaneously. Unfortunately, no existing quantum-safe DPRF simultaneously achieves adaptive security and the desired notions of efficiency as outlined above. To illustrate this, we summarize known instances of plausibly post-quantum DPRFs and their properties below.

DPRF from any PRF. The known construction of DPRF from any PRF [30] is adaptively secure, but incurs *both* exponential key storage *and* exponential online overheads for encryption/decryption. This construction essentially captures the schemes which perform an MPC (Multi-Party Computation)-based evaluation of any PRF.

DPRF from Threshold FHE. One can build a DPRF in a generic manner from any PRF and a threshold FHE scheme, which acts as a *universal thresholdizer* [10]. Based on known instances of threshold FHE [10, 16, 27, 18, 12], such a construction incurs exponential key storage, but linear online overheads for encryption/decryption. Unfortunately, all known threshold FHE schemes are only statically secure, and hence the resulting DPRF is not adaptively secure. Additionally, the universal thresholdization approach requires non black-box evaluation of the PRF under the hood of FHE, which renders the resulting DPRF practically inefficient, and unsuitable for applications of DiSE.

DPRF from LWE [24]. A concrete construction of adaptively secure DPRF from the LWE assumption was presented in [24]. This scheme incurs exponential key storage, but linear overheads for online evaluation. Unfortunately, this scheme requires a *super-polynomial modulus-to-noise ratio*, and involves a log-depth (in the size of the input) evaluation circuit (aka Naor-Reingold [31]) consisting of several matrix multiplications. Hence, this scheme is not amenable to practically efficient realizations of DiSE.

DPRF from LWR. Recently, the authors of [35] proposed a practically efficient DPRF construction based on LWR and a corresponding instantiation of DiSE, called PQDiSE. PQDiSE incurs exponential key storage, but linear online overheads for encryption/decryption. Unfortunately, their construction is only proven to be statically secure. While adaptive security may be achieved by naïvely guessing the corrupted set a priori, this leads to an exponential blow-up in the online overheads for encryption/decryption, thus compromising on efficiency.

In this paper, we ask the following question:

Can we design a practically efficient, adaptively secure, and plausibly post-quantum instance of DiSE?

Concretely, we ask if it is possible to design a practically efficient and adaptively secure DPRF from lattice-based hardness assumptions, with linear online evaluation overheads, while avoiding the requirement of a super-polynomial modulus-to-noise ratio as in [24].

DPRFs	Key Share Size	Partial Evaluation Overheads	Total Evaluation Overheads	Post-Quantum?	Adaptively Secure?
AES(128)-based (combinatorial) [2]	$O(\binom{T-1}{t-1})$	$O(\binom{T-1}{t-1})$	$O(\binom{T-1}{t-1})$	No	Yes (small t, T)
DDH-based [29]	$O(1)$	$O(1)$	$O(t)$	No	Yes
ThFHE-based [10]	$O(\binom{T-1}{t-1})$	$O(1)$	$O(t)$	Yes	No
LWE-based [24]	$O(\binom{T-1}{t-1})$	$O(\lambda t)$	$O(\lambda t)$	Yes	Yes
LWR-based [35]	$O(\binom{T-1}{t-1})$	$O(1)$	$O(t)$	Yes	No
MLWR-based (Ours)	$O(\binom{T-1}{t-1})$	$O(1)$	$O(t)$	Yes	Yes

Table 1: Asymptotic comparison of DPRFs for λ-bit inputs (λ being the security parameter). Note that for the adaptively secure LWE-based DPRF from [24], the factor of λ in the overheads for partial and total evaluation reflects the requirement of $O(\lambda)$ matrix multiplications for every partial evaluation. In comparison, our DPRF requires a constant number of polynomial multiplications for each partial evaluation.

1.1 Our Contributions

We answer the above question in the affirmative. Our contributions are as summarized below.

Adaptive DPRF from Module LWR. We introduce an adaptively secure DPRF from the Module Learning with Rounding (MLWR) assumption [14, 8, 22] with polynomial modulus-to-noise ratio. We prove the adaptive security of our construction in the Quantum Random Oracle model (QROM). Our construction supports arbitrary threshold access structures and incurs exponential key storage but linear online evaluation overheads, thus matching the asymptotic efficiency of the best lattice-based non-adaptive DPRFs. In terms of online evaluation time, our construction outperforms *all* existing adaptively secure DPRF constructions.

Table 1 compares our proposed DPRF against existing DPRFs for λ-bit inputs (λ being the security parameter). Except [24], all prior constructions are either quantum-broken, or lack adaptive security, or suffer from inefficient online evaluation. In the case of [24], the factor of λ in the evaluation overheads reflects that each partial evaluation requires $O(\lambda)$ matrix multiplications, where the matrix dimensions are similar to the the degree of a polynomial in our MLWR-based DPRF. In contrast, our MLWR-based DPRF only requires a constant number of polynomial multiplications for each partial evaluation, and is significantly more efficient in practice.

EQuADiSE. Our proposed DPRF yields a realization of DiSE, which we call EQuADiSE (**E**fficient and **Qu**antum-safe **A**daptive **Di**stributed **S**ymmetric-key **E**ncryption). EQuADiSE is the first practically efficient, adaptively secure, and plausibly post-quantum instance of DiSE. In particular, EQuADiSE is the first adaptively post-quantum secure instance of DiSE that incurs linear (in the number of parties) encryption/decryption overheads.

Experimental Validation. In Section 5, we experimentally evaluate the practical efficiency of our proposed MLWR-based DPRF and EQuADiSE, and compare these against all of the DPRFs and the corresponding realizations of DiSE from Table 1. Our results show that our MLWR-based DPRF is faster than all other DPRFs (including the ones based on AES-128 and DDH) in terms of online evaluation time, particularly for larger values of (t, T) (where T denotes the total number of parties and t denotes the threshold). The naïve distributed construction of AES-based DPRF is essentially equivalent to a generic MPC-based technique of evaluating AES (which is viewed as a PRF). Thus, our construction is more efficient than a generic MPC-based solution of DPRF. These efficiency benefits also translate to EQuADiSE, which achieves higher online throughput than *all* prior realizations of DiSE, and scales smoothly to larger values of (t, T). Our implementations of DPRFs and EQuADiSE are available here[1].

1.2 Technical Overview

In this section, we present an overview of the techniques underlying our adaptively secure MLWR-based DPRF.

PRF from LWR. The starting point for our proposed DPRF is a simple *weak* PRF $F : \mathbb{Z}_q^n \times \mathbb{Z}_q^n \to \mathbb{Z}_p$ for appropriately chosen moduli q and p such that $p < q$:

$$F\left(\mathbf{k} \in \mathbb{Z}_q^n, \mathbf{y} \in \mathbb{Z}_q^n\right) = \lfloor \langle \mathbf{y}, \mathbf{k} \rangle \rceil_p,$$

where the operator $\lfloor \cdot \rceil_p$ takes as input an element in $\mathbb{Z}_q$, and rounds it to the nearest integer in $\mathbb{Z}_p$ (in other words, effectively drops the least significant $(\log q - \log p)$ bits). The above is, by definition, a weak PRF under the LWR assumption, for appropriately chosen q, p. Recall that a weak PRF only satisfies pseudorandomness for uniformly sampled inputs. However, one can turn the above weak PRF into a full-fledged PRF in a straightforward manner by applying a random oracle on the input. While our formal treatment in Section 3 considers the full-fledged random oracle-based PRF, we focus on the weak PRF setting in the overview for ease of exposition. This setting suffices to provide a flavor of our core techniques.

Non-Adaptive DPRF from LWR [35]. In [35], it was shown that the above PRF can be extended to a (t, T)-DPRF for arbitrary (reconstruction and privacy) threshold $t < T$ with security against *static corruptions*. Their approach was to secret share the key $\mathbf{k}$ among T parties using *Linear Integer Secret Sharing* (LISS) [19] in the setup phase, such that any subset of t parties can linearly reconstruct the key. Distributed evaluation of the PRF on an input $\mathbf{y}$ is achieved by a t-sized subset $\mathcal{S}$ of parties as follows: first each party $P_i \in \mathcal{S}$ uses its key share $\mathbf{k}_i$ to compute $z_i = \lfloor \langle \mathbf{y}, \mathbf{k} \rangle \rceil_{q_1}$, where q_1 is an additional intermediate modulus such that $p < q_1 < q$. Finally, the evaluator obtains the final output as $z^* = \lfloor f(\{z_i\}_{P_i \in \mathcal{S}}) \rceil_p$, where f is some linear function of the partial evaluations $\{z_i\}$ (see Section 3.1 for a more formal exposition).

The authors of [35] proved that the above construction is a (weak) DPRF that satisfies both consistency and security against static corruptions of up to $(t-1)$ parties for appropriately chosen $q, q_1, p = \text{poly}(\lambda)$ (λ being the security parameter) assuming the hardness of LWR for polynomial modulus-to-modulus ratio [8]. At a high level, their proof strategy crucially uses the knowledge of the corrupt set of parties and the corresponding key shares (in the case of static corruptions, this is known at setup) to construct a simulator that simulates an honest party's partial evaluation on any uniformly random input.

[1] `https://github.com/SayaniSinha97/EQuADiSE`

Adaptive Security of Above DPRF in ROM. Our first contribution is in proving that the above DPRF is, in fact, secure against *adaptive corruptions* of up to $(t-1)$ parties for appropriately chosen $q, q_1, p = \text{poly}(\lambda)$, while relying on the same assumption of the hardness of LWR for polynomial modulus-to-modulus ratio. Note that, in the case of adaptive corruptions, the set of corrupt parties is not known from beforehand, and hence, one cannot follow the simulation strategy from [35] of using the knowledge of the corrupt parties' secret shares to simulate the honest party's partial evaluations. In particular, consider a scenario where the adversary adaptively corrupts an honest party P_i and learns its secret share. In this case, all simulated partial evaluations corresponding to P_i with respect to prior queries must be consistent with the revealed secret share once the adversary corrupts P_i. It is not immediately clear how to do this since the simulator does not learn P_i's key share until the adversary chooses to corrupt it.

To address this, we present an alternative proof strategy that relies on the programmability of the random oracle. At a high level, we show that the simulator is able to perfectly simulate all the responses to the adversary's queries without knowing any of the actual key shares. This effectively implies that no meaningful information about the underlying secret shares is leaked to the adversary through the query-response phase in the real world. To achieve this, the simulator samples one uniform random matrix $\mathbf{A}$ over $\mathbb{Z}_q^{n \times m}$, and another uniform random matrix $\mathbf{R}$ over $\mathbb{Z}_q^{n \times n}$. Let $\mathcal{H}(\cdot)$ be a random oracle that is expected to return a uniformly random $\mathbf{y}$ over $\mathbb{Z}_q^n$ for any query input x.

- For each new query input x, the random oracle response $\mathcal{H}(x)$ is simulated with $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}$ for a newly sampled $\mathbf{s} \xleftarrow{\$} \{0,1\}^m$. The value $\mathcal{H}(x) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}$ represents a random subset-sum of m columns of $(\mathbf{R} \cdot \mathbf{A})$, and due to the leftover hash lemma [32], $\mathcal{H}(x)$ is uniformly random over $\mathbb{Z}_q^n$ for sufficiently large m.
- The response to the partial evaluation query for input x with respect to an uncompromised share $\mathbf{k}_j$ is simulated without using the knowledge of $\mathbf{k}_j$ at all. Basically, a uniform random $\mathbf{k}'_j \xleftarrow{\$} \mathbb{Z}_q^n$ is sampled and the simulated response is $\lfloor \langle \mathbf{A} \cdot \mathbf{s}, \mathbf{k}'_j \rangle \rceil_{q_1}$. The value $\mathbf{s}$ corresponds to the input x, as specified during a random oracle query for input x. Since $\mathbf{A} \cdot \mathbf{s}$ is uniformly random over $\mathbb{Z}_q^n$ due to the leftover hash lemma [32] and so is $\mathbf{k}'_j$, the distribution of the simulated partial evaluations exactly matches with that of the real ones.
- At a later point in the game, when it is time to reveal the share $\mathbf{k}_j$ to the adversary, (i.e., the adversary compromises $\mathbf{k}_j$), the simulator cheats the adversary by revealing $\mathbf{k}''_j$ instead of $\mathbf{k}_j$. Here, $\mathbf{k}''_j$ is computed as $\mathbf{R}^{-1} \cdot \mathbf{k}'_j$. It is important to note that,

  $$\lfloor \langle \mathcal{H}(x), \mathbf{k}''_j \rangle \rceil_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}, \mathbf{k}'_j \rangle \rceil_{q_1},$$

  which implies that the simulated response to the corruption query ($\mathbf{k}''_j$) remains consistent with the previous responses to partial evaluation queries corresponding to $\mathbf{k}_j$, from the point of view of the adversary.

Clearly, for a feasible computation of $\mathbf{k}''_j$, $\mathbf{R}$ must be invertible modulo q. This can be taken care of during the setup of the random oracle while sampling a suitable $\mathbf{R}$.

We discuss the detailed proof of adaptive security in Section 3.3 involving additional techniques.

Security in the Quantum Random Oracle Model. In practice, a random oracle is instantiated by a fixed function, typically a cryptographic hash function. Hence, quantum-safe

analyses in the random oracle model should take into consideration adversaries with quantum oracle access [9]. However, the LWR-based DPRF in [35] proves security only against adversaries with classical access. This notion is weaker, since quantum adversaries can query the oracle in superposition, thereby accessing exponentially many inputs per query and potentially extracting more information than any polynomial number of classical queries allows. We show that our proof of adaptive security for the DPRF construction of [35] holds even in the quantum random oracle model (QROM), while facilitating the adversary with negligible amount of additional distinguishing advantage.

Adaptation to Module LWR. We present a more practically efficient variant of the above-discussed DPRF by switching from the LWR assumption over integer lattices to the Module LWR (MLWR) assumption over module lattices. We note that several NIST standards, notably ML-KEM[1] and ML-DSA[2]) rely on Module LWE/LWR for better efficiency as compared to their LWE/LWR-based counterparts, and we are not aware of any attacks that perform substantially better against these assumptions as compared to LWE/LWR.

Our starting point is the natural adaptation of the LWR-based weak PRF outlined above to the setting of MLWR:

$$F(\mathbf{K} \in \mathcal{R}_q^d, \mathbf{X} \in \mathcal{R}_q^d) = \lfloor \mathbf{X} \cdot \mathbf{K} \rceil_p \in \mathcal{R}_p$$

where $\mathcal{R}_q$ and $\mathcal{R}_p$ are appropriately chosen polynomial rings, p is an appropriate rounding modulus, and d is the module rank. We then apply the same two-step rounding technique to extend this weak PRF into a weak DPRF. To establish the adaptive security of the MLWR-based DPRF, we adapt our techniques for proving the adaptive security of the LWR-based DPRF with only one key modification, namely, the use of the regularity lemma from [26] instead of the leftover hash lemma [32] to accommodate the module lattice framework. We refer to Section 3.3 for the detailed construction and adaptive proof of security.

2 Preliminaries

2.1 Notations

The notation $x \leftarrow \mathcal{X}$ signifies that x is sampled from the distribution $\mathcal{X}$, whereas $x \xleftarrow{\$} X$ means that, x is sampled uniformly at random from the set X. For $q, N \in \mathbb{N}$, $\mathcal{R}_{N,q} = \mathbb{Z}_q[x]/(x^N + 1)$ represents a ring of polynomials with coefficients in $\mathbb{Z}_q$ and degree at most $(N-1)$. Bold upper case (e.g., $\mathbf{A}$) variable denotes either a vector of polynomials over $\mathcal{R}_{N,q}$ or a matrix in $\mathbb{Z}_q$, depending upon the context. With two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^n$, $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{n} a_i b_i \pmod q$ represents their vector dot product modulo q. With two polynomials $A, B \in \mathcal{R}_{N,q}$, $(A \cdot B) \in \mathcal{R}_{N,q}$ denotes polynomial multiplication modulo $(x^N + 1)$. Provided two vectors of polynomials $\mathbf{A}$, $\mathbf{B} \in \mathcal{R}_{N,q}^d$, $\mathbf{A} \cdot \mathbf{B}$ denotes $\sum_{i=1}^{d} \mathbf{A}_i \mathbf{B}_i \pmod{(x^N + 1)} \in \mathcal{R}_{N,q}$. The cardinality of a set S is denoted by $|S|$. The notation $[n]$ for some $n \in \mathbb{N}$ denotes the set $\{1, \ldots, n\}$. For any $y \in \mathbb{Z}_q$, the round-off operation, denoted by $\lfloor y \rceil_p$ gives the nearest integral value of $(y \cdot \frac{p}{q})$ in $\mathbb{Z}_p$; in particular, if $(y \cdot \frac{p}{q})$ has a fractional part exactly equal to 0.5, we choose to always round it down to $\lfloor y \cdot \frac{p}{q} \rfloor$ to avoid ambiguity. We can apply the round-off operator to vectors, matrices and polynomials as well to denote element-wise/coefficient-wise round-off operation. For security parameter λ, $\mathrm{negl}(\lambda)$ and $\mathrm{poly}(\lambda)$ denote a negligible function and a polynomial function of λ respectively.

[1] `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf`
[2] `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf`

2.2 Some Terminologies and Definitions

Threshold Access Structure. Let $\mathcal{P} = \{P_1, \ldots, P_T\}$ be a set of T parties, and suppose that some secret k is distributed among them in form of secret shares. Access structure is a set consisting of all "valid" subsets $\overline{\mathcal{P}} \subseteq \mathcal{P}$ of parties that can recover the secret k by combining their key shares together. For any $t, T \in \mathbb{N}$ $(t \leq T)$, a minimal (t, T)-threshold access structure over $\mathcal{P}$ is defined as a collection of valid subsets of the form $\mathbb{A}_{t,T} = \{\overline{\mathcal{P}} \subseteq \mathcal{P} : \left|\overline{\mathcal{P}}\right| = t\}$, such that we have $|\mathbb{A}_{t,T}| = \binom{T}{t}$ as the number of valid subsets.

Monotone Boolean Formula (MBF). A Boolean formula is monotone if it has a single output and it consists of only AND and OR combination of Boolean variables. Note that any (t, T)-threshold access structure $\mathbb{A}_{t,T}$ can be represented by a MBF. The fact that a particular collaboration of t parties is able to reconstruct the secret, is captured by Boolean formula of the form $(x_1 \wedge \ldots \wedge x_t)$ and that any such t-collaboration is a way of reconstruction, is captured by ORing $\binom{T}{t}$ such terms. For e.g., $\mathbb{A}_{3,4}$ is represented by $(x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge x_2 \wedge x_4) \vee (x_1 \wedge x_3 \wedge x_4) \vee (x_2 \wedge x_3 \wedge x_4)$.

Pseudo Random Function (PRF). We recall the formal definition of a pseudorandom function (PRF). Let $\mathcal{F} : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ be a family of pseudo random functions and $\mathcal{F}' = \{f' | f' : \mathcal{X} \to \mathcal{Y}\}$ be the set of all possible functions with the same domain and range. Let us assume that, $f_k \in \mathcal{F}$ uses a uniform random secret $k \xleftarrow{\$} \mathcal{K}$ and, on input $x \in \mathcal{X}$, outputs $f_k(x)$, using both k and x. Then, the advantage of any PPT (Probabilistic Polynomial Time) distinguisher $\mathcal{D}$ is negligible, i.e.,

$$\left|\mathsf{Pr}[\mathcal{D}^{f_k(\cdot)}(1^\lambda) = 1] - \mathsf{Pr}[\mathcal{D}^{f'(\cdot)}(1^\lambda) = 1]\right| < \mathrm{negl}(\lambda),$$

where λ is a security parameter. The first probability is taken over uniform choice of k and randomness of $\mathcal{D}$, and the second probability is taken over uniform choice of f' and randomness of $\mathcal{D}$.

Learning with Rounding (LWR) problem. LWR problem, first introduced in [6], is a "derandomized" version of Learning with Errors (LWE) problem [32]. Given a parameter $n \in \mathbb{N}$, two moduli $q, p \in \mathbb{N}$ such that $q > p \geq 2$, the LWR distribution $L_{\mathbf{s}}$ for a secret $\mathbf{s} \in \mathbb{Z}_q^n$ is defined over $\mathbb{Z}_q^n \times \mathbb{Z}_p$ of the form $(\mathbf{a}, b)$, where we choose $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ and then calculate $b = \lfloor \langle \mathbf{a}, \mathbf{s} \rangle \rceil_p$.
The decision LWR problem is to efficiently distinguish samples of $L_{\mathbf{s}}$ from uniformly random sample of $\mathbb{Z}_q^n \times \mathbb{Z}_p$, and it is assumed to be hard for proper choice of n, q and p.

Module Learning with Rounding (MLWR) problem. This is a variant of LWR problem defined on module lattice, which is a structured lattice, however having lesser structure than an ideal lattice. Module lattice essentially bridges the gap between unstructured lattice and ideal lattice, while preserving the quality of both: hardness of problems defined on unstructured lattice and efficient parameter choices of hard problems defined on ideal lattice. LWR problem when defined on ideal lattice, is called ring-LWR problem and when defined on module lattice, is called module-LWR problem, just similar to its well-studied LWE (Learning with Errors) counterpart. Concept of module LWE was first introduced in [14] and its relation to worst-case hard problems defined on module lattice was formally analyzed in [23]. Several NIST candidates for post-quantum secure cryptosystems [22] rely on the hardness of the MLWR problem. The assumption is as follows.

Let $\mathbf{K} \in \mathcal{R}_q^d$ be the fixed MLWR secret. Then an MLWR sample with secret $\mathbf{K}$ is of the form $(\mathbf{A}, B)$, where $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^d$ and $B = \lfloor \mathbf{A} \cdot \mathbf{K} \rceil_p$. Here, $\mathbf{A} \cdot \mathbf{K}$ denotes $\sum_{i=1}^{d} \mathbf{A}_i \cdot \mathbf{K}_i$ such that $\mathbf{A}_i \cdot \mathbf{K}_i$ is

the polynomial multiplication modulo $(x^N + 1)$. The MLWR assumption is that distinguishing MLWR sample from a truly random sample of $\mathcal{R}_q^d \times \mathcal{R}_p$ efficiently is hard for a PPT adversary with proper choice of parameters.

2.3 (t, T)-Threshold Secret Sharing

A threshold secret sharing scheme is an essential underlying primitive to build a distributed PRF protocol. A (t, T)-threshold secret sharing scheme shares a key k among these T parties in such a way that any t or more parties are able to reconstruct it from their respective shares, though collaboration of less than t parties does not suffice. We choose to use Benaloh-Leichter Linear Integer Secret Sharing Scheme (LISSS) as described in [19]. The secret sharing scheme is "linear integer" because key shares can be linearly combined to get the actual secret back in a way that the coefficients of the linear combination are integers. These coefficients used during the reconstruction of the secret are called *recovery coefficients*. Though the original Benaloh-Leichter LISSS shares a scalar secret, it can naturally be extended to share a secret in vector form. As we deal with secrets belonging to $\mathbb{Z}_q^n$ in later sections, we describe the LISSS scheme in the context of sharing a secret vector $\mathbf{k} \in \mathbb{Z}_q^n$ here.

2.3.1 Preprocessing.

Here, we discuss some necessary preprocessing steps for threshold secret sharing.

Formation of distribution matrix M: Formation of distribution matrix $\mathbf{M}$ depends upon the MBF, representing a (t, T)-threshold access structure. As any MBF is a combination of AND and OR of Boolean variables, we need to focus on the three following cases.
Each Boolean variable x_i corresponds to a singleton matrix with 1 as its only element.
AND-ing of $\mathbf{M_{f_a}}$ *and* $\mathbf{M_{f_b}}$*:* Let $\mathbf{M_{f_a}}$ with dimension $d_a \times e_a$ and $\mathbf{M_{f_b}}$ with dimension $d_b \times e_b$ be the distribution matrices for Boolean formulae f_a and f_b respectively. Then we form $\mathbf{M_{f_a \wedge f_b}}$ as follows:

c_a	c_a	C_a	0
0	c_b	0	C_b

Here, c_a and c_b denote the first column of $\mathbf{M_{f_a}}$ and $\mathbf{M_{f_b}}$ respectively. C_a and C_b denote the rest of the columns of $\mathbf{M_{f_a}}$ and $\mathbf{M_{f_b}}$ respectively. $\mathbf{M_{f_a \wedge f_b}}$ has dimension $(d_a + d_b) \times (e_a + e_b)$.
OR-ing of $\mathbf{M_{f_a}}$ *and* $\mathbf{M_{f_b}}$*:* Let $\mathbf{M_{f_a}}$ with dimension $d_a \times e_a$ and $\mathbf{M_{f_b}}$ with dimension $d_b \times e_b$ be the distribution matrices for Boolean formulae f_a and f_b respectively. Then we form $\mathbf{M_{f_a \vee f_b}}$ as follows:

c_a	C_a	0
c_b	0	C_b

Here, c_a and c_b denote the first column of $\mathbf{M_{f_a}}$ and $\mathbf{M_{f_b}}$ respectively. C_a and C_b denote the rest of the columns of $\mathbf{M_{f_a}}$ and $\mathbf{M_{f_b}}$ respectively. $\mathbf{M_{f_a \vee f_b}}$ has dimension $(d_a + d_b) \times (e_a + e_b - 1)$.
It can be easily verified that, the distribution matrix $\mathbf{M}$ for (t, T)-threshold secret sharing has dimension $d \times e$, where $d = \binom{T}{t} t$ and $e = (1 + \binom{T}{t}(t - 1))$.

Formation of share matrix $\boldsymbol{\rho}$: $\boldsymbol{\rho}$ is a matrix with dimension $e \times n$. Its first row is populated from the n elements of the actual secret vector $\mathbf{k} \in \mathbb{Z}_q^n$. The rest of the elements of the matrix are filled uniformly randomly from $\mathbb{Z}_q$.

2.3.2 Sharing.

First we compute the matrix $\mathbf{M} \cdot \boldsymbol{\rho}$ that has $d = \binom{T}{t}t$ rows. Each of the rows is a unique key share. Note that the number of t-sized subset of $\mathcal{P}$ is $\binom{T}{t}$ and each of the t parties in a t-sized subset will hold a keyshare corresponding to that specific group, which justifies $d = \binom{T}{t}t$ to be the total number of unique keyshares. For ease of explanation, we identify each keyshare with the following two attributes: (1) party_id (which party the key share belongs to), (2) group_id (which t-sized group the key share is used for). By enumerating over all t-sized subsets and tagging them with corresponding enumerating serial numbers, we get group_id's of all t-sized subsets.

Now sharing of d rows among T parties happens in the following manner: we consider $d = \binom{T}{t}t$ rows as $\binom{T}{t}$ chunks of rows of size t. Now for $i \in [\binom{T}{t}]$, we pick i^{th} such chunk at a time and assign each of the t rows to parties belonging to the subset with group_id i. For example, in a $(3, 5)$-threshold secret sharing, subset $\{P_1, P_2, P_3\}$ has group_id 1, so first three rows of $\mathbf{M}\boldsymbol{\rho}$ are assigned to P_1, P_2, P_3 respectively. $\{P_1, P_2, P_4\}$ has group_id 2, so next three rows of $\mathbf{M} \cdot \boldsymbol{\rho}$ are assigned to P_1, P_2, P_4 respectively and so on.

2.3.3 Reconstruction.

Any t-sized group of parties, with their key shares, should be able to reconstruct $\mathbf{k}$. Given $\mathcal{P}' = \{P_{i_1}, P_{i_2}, \ldots, P_{i_t}\} \subset \mathcal{P}$ with $i_1 < \cdots < i_t$, each of the t parties will have one key share with group_id corresponding to $\mathcal{P}'$. Let us denote these t key shares as $\{\mathbf{k}_{i_1}, \ldots, \mathbf{k}_{i_t}\}$. Without loss of generality, in any t-sized group, the party with minimum value of party_id is called the group_leader. Hence, P_{i_1} is the group_leader here. In this LISSS the recovery coefficient is 1 for the group_leader and -1 for the rest of the $(t-1)$ parties. The key $\mathbf{k}$ can be reconstructed as $\mathbf{k} = \mathbf{k}_{i_1} - \sum_{j=2}^{t} \mathbf{k}_{i_j}$. We exploit this reconstruction property in final evaluation of (t, T)-threshold PRF.

A Toy Example. We summarize the precise output of the above-mentioned threshold secret sharing scheme with the help of a toy example here. We consider modulus $q = 8$ and dimension $n = 3$. Let the secret be $\mathbf{k} = [2, 1, 6]$ and we apply $(3, 4)$-threshold secret sharing on $\mathbf{k}$ to distribute it among parties $\{P_1, P_2, P_3, P_4\}$. We deal with the straight-forward MBF for $(3, 4)$-threshold, which is, $f = x_1x_2x_3 + x_1x_2x_4 + x_1x_3x_4 + x_2x_3x_4$. After performing the above-mentioned preprocessing and sharing procedure, the matrix $\mathbf{M} \cdot \boldsymbol{\rho}$ will have $\binom{4}{3} \cdot 3 = 12$ rows and $n = 3$ columns. Each row of $\mathbf{M} \cdot \boldsymbol{\rho}$ is a secret share and is an element of $\mathbb{Z}_8^3$. If the i^{th} literal in the MBF is x_j, then party P_j gets the i^{th} row of the matrix as a secret share. Effectively, each of the four 3-sized subsets have a $(3, 3)$-linear integer secret sharing within it. A hypothetical value of matrix $\mathbf{M} \cdot \boldsymbol{\rho}$ is the following:

$$\begin{bmatrix} 0 & 4 & 1 \\ 2 & 1 & 4 \\ 4 & 2 & 7 \\ 2 & 1 & 3 \\ 3 & 2 & 3 \\ 5 & 6 & 2 \\ 5 & 2 & 3 \\ 2 & 6 & 0 \\ 1 & 3 & 5 \end{bmatrix}$$

Let us pick one 3-sized subset $\{P_1, P_2, P_4\}$ whose group_id is 2. Following the rule of sharing, P_1, P_2, P_4 get 4^{th},5^{th} and 6^{th} row of $\mathbf{M} \cdot \boldsymbol{\rho}$, and we denote them with $\mathbf{k}_{2,1}$, $\mathbf{k}_{2,2}$ and $\mathbf{k}_{2,4}$ respectively. The two subscripts of each share denote the group_id and party_id respectively. P_1 is the group_leader of the group $\{P_1, P_2, P_4\}$. The secret $\mathbf{k}$ is reconstructed as,

$$\mathbf{k} = \mathbf{k}_{2,1} - \mathbf{k}_{2,2} - \mathbf{k}_{2,4} = ([2,1,3] - [3,2,3] - [5,6,2]) \mod 8 = [2,1,6].$$

In short, P_1 has to store $\mathbf{k}_{1,1}$, $\mathbf{k}_{2,1}$, $\mathbf{k}_{3,1}$ corresponding to subsets $\{P_1, P_2, P_3\}$, $\{P_1, P_2, P_4\}$, $\{P_1, P_3, P_4\}$ respectively, P_2 has to store $\mathbf{k}_{1,2}$, $\mathbf{k}_{2,2}$, $\mathbf{k}_{4,2}$ corresponding to subsets $\{P_1, P_2, P_3\}$, $\{P_1, P_2, P_4\}$, $\{P_2, P_3, P_4\}$ respectively, and so on.

Size of Secret Shares. After applying (t, T)-threshold secret sharing on $\mathbf{k} \in \mathbb{Z}_q^n$, each party gets $\binom{T-1}{t-1}$ key shares to store. So each party has to store $\binom{T-1}{t-1} \cdot n \cdot \lceil \log_2 q \rceil$ bits in total.

Choice of LISS over Shamir's secret sharing. We use LISS for sharing the PRF secret instead of the well known Shamir's secret sharing [33], since reconstruction of the secret requires only $\{-1, 1\}$-integer linear combination of the shares in the former case while in the latter case, much larger Lagrange coefficients serve as the recovery coefficients. Hence, the rounding error from each party gets multiplied by the Lagrange coefficients and it would require even larger (practically inefficient) $\frac{q_1}{p}$ ratio in order to eliminate the errors in the final rounding phase.

2.4 Distributed PRF (DPRF)

Distributed PRF supports distributed evaluation of the PRF by multiple parties. The PRF secret k always remain distributed in form of secret shares among multiple parties. Following is a formal definition of distributed PRF.

Definition 1 (Distributed Pseudo Random Function (DPRF))**.** *Let $\mathcal{P} = \{P_j : j \in [T]\}$ be a set of T parties. A (t, T)-distributed PRF* DPRF *with input space $\mathcal{X}$, key space $\mathcal{K}$, and output space $\mathcal{Y}$ consists of three PPT algorithms as described below.*

$$\mathsf{DPRF} = (\mathsf{DPRF.Setup}, \mathsf{DPRF.PartialEval}, \mathsf{DPRF.FinalEval}).$$

$\mathsf{DPRF.Setup}(1^\lambda, \mathbb{A}_{t,T})$*: On input the security parameter λ and threshold access structure $\mathbb{A}_{t,T}$, total number of parties T, and threshold number of parties t, this algorithm generates a key $k \xleftarrow{\$} \mathcal{K}$, and then shares it among T parties using some t-out-of-T threshold secret sharing scheme. At the end of the secret sharing process, the actual key k is not stored anywhere. Each party has to store one or more than one key share(s) depending on the particular threshold secret sharing scheme used.*

$\mathsf{DPRF.PartialEval}(x, P_j, \mathcal{P}')$*: On input a valid subset $\mathcal{P}' \subset \mathcal{P}$ with $|\mathcal{P}'| = t$, an input $x \in \mathcal{X}$ and a party $P_j \in \mathcal{P}'$, the appropriate key share (say, k_j) of P_j corresponding to $\mathcal{P}'$ is chosen and a partial evaluation $f_{k_j}(x)$ is returned. Note that, if $|\mathcal{P}'| \geq t$, we consider any t parties among them to get a minimal valid subset and proceed in the same manner.*

$\mathsf{DPRF.FinalEval}(\mathcal{P}', \{f_{k_j}(x)\}_{P_j \in \mathcal{P}'})$*: On input a valid subset $\mathcal{P}' \subset \mathcal{P}$ with $|\mathcal{P}'| = t$, and all the partial evaluations by parties $P_j \in \mathcal{P}'$, this algorithm combines them to get the final PRF evaluation. The actual combination procedure depends upon the reconstruction property of the underlying threshold secret sharing scheme.*

Correctness and Consistency of Distributed PRF. A (t, T)-distributed PRF with $f_k(\cdot)$ as its underlying PRF is *correct* if given an input, its distributed evaluation by any valid subset $\mathcal{P}' \subset \mathcal{P}$ outputs the same value as would be obtained by directly evaluating $f_k(\cdot)$ on the same input except with a negligible probability, i.e.,

$$\mathsf{Pr}[\mathsf{DPRF.FinalEval}(\mathcal{P}', \{\mathsf{DPRF.PartialEval}(x, P_j, \mathcal{P}')\}_{P_j \in \mathcal{P}'}) = f_k(x)] \geq 1 - \mathrm{negl}(\lambda).$$

A (t, T)-distributed PRF is *consistent* if distributed evaluation on a given input by any two distinct valid subsets $S_1, S_2 \subset \mathcal{P}$ with $|S_1| = |S_2| = t$ outputs the same value except with a negligible probability, i.e.,

$$\big|\mathsf{Pr}[\mathsf{DPRF.FinalEval}(S_1, \{\mathsf{DPRF.PartialEval}(x, P_j, S_1)\}_{P_j \in S_1}) \\ \neq \mathsf{DPRF.FinalEval}(S_2, \{\mathsf{DPRF.PartialEval}(x, P_{j'}, S_2)\}_{P_{j'} \in S_2})]\big| \leq \mathrm{negl}(\lambda).$$

Note that the correctness of (t, T)-distributed PRF implies its consistency, but not the other way around.

Static Security of Distributed PRF. Following is the notion of static security of DPRF from [30].

The adversarial model. We assume a PPT adversary that can statically corrupt (i.e., announces the set of corrupt parties before the partial evaluation query phase starts) at most $(t-1)$ number of parties and each party if corrupted, is honest but curious; i.e., no corrupted party violates the protocol of the construction but tries to gather information about unknown secret share in all possible probabilistic ways within polynomial time.

The security notion. Let $\mathcal{P} = \{P_j : j \in [T]\}$ be the set of T parties and $\mathcal{A}$ be a PPT adversary as described above. Let $\widehat{\mathcal{P}}$ be a statically corrupted set such that, $|\widehat{\mathcal{P}}| = (t-1)$. Hence, after $\mathsf{DPRF.Setup}(1^\lambda, t, T)$ is run, $\mathcal{A}$ has access to key shares of each $P_j \in \widehat{\mathcal{P}}$. We say that DPRF is secure if the winning probability of $\mathcal{A}$ against a challenger $\mathcal{C}$ in the following game is negligible.

Game

Partial Evaluation Query: $\mathcal{A}$ sends a query input $x \in \mathcal{X}$ to $\mathcal{C}$. In response, $\mathcal{C}$ sends to $\mathcal{A}$ $(f_k(x), \{f_{k_j}(x)\}_{P_j \in \mathcal{P} \setminus \widehat{\mathcal{P}}})$, where $f_{k_j}(x) = \mathsf{DPRF.PartialEval}(x, P_j, \widehat{\mathcal{P}} \bigcup P_j)$.

Challenge Phase: $\mathcal{A}$ sends a new challenge query $x^\star$ (different from query phase inputs) to $\mathcal{C}$. Now, $\mathcal{C}$ chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. If $b = 0$, it sends $f_k(x^\star)$ to $\mathcal{A}$, otherwise, it sends some $y \xleftarrow{\$} \mathcal{Y}$ to $\mathcal{A}$.

The adversary can raise a polynomial (in security parameter λ) number of partial evaluation queries before the challenge phase. After $\mathcal{A}$ sees the output corresponding to challenge input $x^\star$, it outputs a distinguishing bit b'. $\mathcal{A}$ wins the game, if $b = b'$. The (t, T)-DPRF is statically secure, if $\mathsf{Pr}[b = b'] \leq \mathrm{negl}(\lambda)$.

Adaptive Security of Distributed PRF. Now, we describe the notion of adaptive security of distributed PRF in the form of a game between a challenger Ch and an adversary Ad.

Adversarial model. In case of a (t, T)-distributed DPRF with any $t < T$, we consider a semi-honest PPT adversary which can adaptively corrupt at most $(t-1)$ number of parties among a total of T parties.

The security notion. Let λ be the security parameter. Let $\mathcal{P} = \{P_j : j \in [T]\}$ be the set of T parties. The challenger first chooses a secret $\mathbf{k} \in \mathcal{K}$. Then, $\mathbf{k}$ is distributed among T parties

using some (t, T)-threshold secret sharing such that knowledge of any $(t-1)$ secret shares does not leak any information about the t^{th} share and hence reconstruction of the secret $\mathbf{k}$ from $(t-1)$ secret shares is information-theoretically hard. The list of corrupted secret shares $\mathcal{L}$ is initially Φ (i.e., empty) at the beginning of the game and gets populated as the following game progresses between Ch and Ad.

Game:

Partial Evaluation Query: Ad adaptively queries Ch with $(x, P_j, \mathcal{P}')$, to see partial evaluations by party P_j on input x with respect to a t-sized group $\mathcal{P}'$, such that $P_j \in \mathcal{P}'$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$. In response, Ch computes the partial evaluation on input x using the secret share of P_j corresponding to t-sized group $\mathcal{P}'$.

Corruption Query: Here, Ad sends corruption query of the form $(\mathcal{P}', P_j)$ to Ch such that $P_j \in \mathcal{P}'$ and $|\mathcal{P}'| = t$. In response, Ch sends the secret share of P_j corresponding to $\mathcal{P}'$ to Ad. The list $\mathcal{L}$ is appended with $(\mathcal{P}', P_j)$. For the same t-sized group $\mathcal{P}'$, the total number of corrupted parties in $\mathcal{L}$ should not exceed $(t-1)$; otherwise, the game is aborted.

Challenge Phase: In this phase, Ad provides Ch with a challenge input $x^\star$. Now, Ch chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. If $b = 0$, it sends the DPRF evaluation by any t parties of $\mathcal{P}$ on challenge input $x^\star$ to Ad. Otherwise, it sends $y^\star \xleftarrow{\$} \mathcal{Y}$, i.e., a truly random output over the range of the PRF, to Ad.

Post-challenge Partial Evaluation Query: This is exactly the same as the pre-challenge partial evaluation query phase.

Post-challenge Corruption Query: This phase is exactly the same as the pre-challenge corruption query phase.

Partial evaluation queries (upper-bounded by $\text{poly}(\lambda)$) and corruption queries can be raised in any order throughout the game. The challenge phase occurs only once and can happen anytime during the game. Finally, Ad outputs a distinguishing bit b'. The distributed PRF is said to be adaptively secure if $\Pr[b' = 1 | b = 0] - \Pr[b' = 1 | b = 1] \leq \text{negl}(\lambda)$.

Static security does not imply adaptive security. Intuitively, it should be already evident that adaptive corruption models a stronger adversary than static corruption. The following theorem formally supports this argument.

Theorem 1 (Adapted from Theorem 2.24 of [24])**.** *For any $t, T \in \text{poly}(\lambda)$, such that $\binom{T}{t-1}$ is super-polynomial, there exists a family of distributed pseudorandom functions, which is secure in the static corruption model but insecure in the adaptive corruption model.*

Interested readers are referred to [24] for a formal proof of the theorem. In a broad sense, the proof shows that if the adversary is allowed to raise even a single partial evaluation query before starting to corrupt parties, its advantage in distinguishing PRF output from truly random output on challenge input increases significantly. Note that, for adaptive corruption in a (T, T)-distributed DPRF, the adversary may correctly guess the corrupted $(T-1)$ parties with probability $\frac{1}{T}$ in the beginning of the protocol. In this case, static security implies adaptive security with a non-negligible probability for polynomially large T. Hence, we focus on adaptive security of (t, T)-threshold scenario for any $t < T$.

3 Adaptively Secure Module-LWR based Distributed PRF

In this section, our final objective is to propose the construction of a quantum-safe (t, T)-distributed PRF (DPRF), which achieves adaptive security for any $t < T$. In Section 3.1, we start with recalling the quantum-safe DPRF construction of [35] from LWR assumption. In Section 3.2, we describe our main contribution, namely, the Module-LWR based DPRF construction. In Section 3.3.2, we prove that this MLWR-based DPRF construction is adaptively secure in QROM. Finally, Section 3.4 proposes a concrete choice of parameters for the practical implementation of LWR-based and MLWR-based adaptive DPRFs.

3.1 Recalling LWR-based DPRF of [35]

Given input space $\mathcal{X} = \{0,1\}^\star$, key space $\mathcal{K} = \mathbb{Z}_q^n$, output space $\mathcal{Y} = \mathbb{Z}_p$, two moduli p, q such that $p < q$, and a hash function $\mathcal{H} : \{0,1\}^\star \to \mathbb{Z}_q^n$ modelled as a random oracle, [35] considers the following function $f : \mathcal{X} \times \mathcal{K} \to \mathcal{Y}$ as the base PRF.

$$f_{\mathbf{k}}^{\mathsf{base}}(\mathbf{x}) = \lfloor \langle \mathcal{H}(\mathbf{x}), \mathbf{k} \rangle \rceil_p.$$

We assume $\mathcal{P} = \{P_j : j \in [T]\}$ is a set of T parties and we summarize (t, T)-distributed version of the above PRF (Construction 2 of [35]) for arbitrary $t, T \in \mathbb{N}$, with $t < T$ in the following.

Setup. It samples the key $\mathbf{k} \xleftarrow{\$} \mathcal{K}$, and distributes it among T parties using Benaloh-Leichter linear integer secret sharing scheme [19] such that any valid subset of $\mathcal{P}$ with cardinality t is able to reconstruct the secret $\mathbf{k}$, and destroys $\mathbf{k}$.

Partial Evaluation. For a minimal valid subset $\mathcal{P}' \subset \mathcal{P}$ (i.e., $|\mathcal{P}'| = t$), without loss of generality, let the key shares of the t parties corresponding to $\mathcal{P}'$ be $\mathbf{k}_1, \cdots, \mathbf{k}_t$ respectively. Let q_1 be an appropriately chosen intermediate modulus such that $p < q_1 < q$. Given input $\mathbf{x}$, each $P_j \in \mathcal{P}'$ (parallelly) computes partial evaluation $\mu_j = \lfloor \langle \mathcal{H}(\mathbf{x}), \mathbf{k}_j \rangle \rceil_{q_1}$ and sends it to a designated evaluator in $\mathcal{P}'$.

Final Evaluation. By the reconstruction property of secret sharing scheme [19], any t-sized subset $\mathcal{P}'$ with shares $\{\mathbf{k}_i\}_{j \in [t]}$ satisfy the following: $\mathbf{k}_1 - \sum_{j=2}^{t} \mathbf{k}_j = \mathbf{k}$. The final output of DPRF is computed from the t partial evaluations of parties in $\mathcal{P}'$ as $f_{\{\mathbf{k}_j\}_{j \in [t]}}^{\mathsf{dist}}(\mathbf{x}) = \lfloor \mu_1 - \sum_{j=2}^{t} \mu_j \rceil_p$.

Remark on Security. This construction from [35] is a (weak) DPRF against static corruptions of up to $(t-1)$ parties for appropriately chosen $q, q_1, p = \mathrm{poly}(\lambda)$, by LWR assumption with polynomial modulus-to-modulus ratio [8]. *Static corruption* means that the adversary declares the corrupted set of (at most $(t-1)$) parties beforehand, hence it is simpler to argue security by showing that the honest parties do not leak information about their respective secret shares through their partial evaluations, and thus the output of DPRF is still pseudorandom in the view of the adversary. At a high level, their proof strategy crucially relies on the knowledge of the shares of corrupted parties to construct a simulator that simulates an honest party's partial evaluation on any uniformly random input. However, it does not consider the aspect of adaptive security and proof in QROM.

Remark on parameter choice. The correctness of this DPRF stems from the appropriate selection of moduli p and q_1, ensuring that $\frac{p}{q_1}$ remains sufficiently small. Its security, in turn,

depends on the careful choice of moduli q_1 and q, maintaining a sufficiently small ratio $\frac{q_1}{q}$, as well as an appropriate value for n, such that the underlying LWR problem remains hard. While proposing an efficient choice of parameters, [35] takes into account only practical attacks against LWR/E problem instead of abiding by the theoretical bounds.

3.2 Construction of Module-LWR based DPRF

This section presents our core technical contribution, i.e., an efficient construction of Module-LWR based DPRF, which we prove to be secure against adaptive corruption in QROM. In this section, consider polynomial size N, module rank d, moduli q, q_1, p, all in $\mathbb{N}$ such that $q > q_1 > p$, and a hash function $\mathcal{H}(\cdot) : \{0,1\}^\star \to \mathcal{R}_{N,q}^d$ modelled as random oracle. We begin by presenting a base (non-threshold) PRF construction derived from the MLWR assumption in Construction 1, followed by its distributed variant in Construction 2.

Construction 1 (Quantum-safe PRF from Module-LWR Assumption)**.** *The evaluation of an MLWR-based quantum-safe PRF at an input point $x \in \mathcal{X}$ with key space $\mathcal{K} = \mathcal{R}_q^d$, input space $\mathcal{X} = \{0,1\}^\star$, output space $\mathcal{Y} = \mathcal{R}_p$ is described as: $f_{\mathbf{K}}^{\mathsf{base}}(x) = \lfloor \mathcal{H}(x) \cdot \mathbf{K} \rceil_p$, where $\mathbf{K} \in \mathcal{K}$ is the fixed secret key of the PRF.*

Since $\mathcal{H}(\cdot)$ is modelled as random oracle, for any arbitrary input x, $\mathcal{H}(x)$ is uniform random over $\mathcal{R}_q^d$. Due to MLWR assumption (Section 2.2), Construction 1 is quantum-safe for appropriate choice of N, d, q, p.

Construction 2 (Quantum-safe (t,T)-Distributed PRF from Module-LWR Assumption)**.** *Let $\mathcal{P} = \{P_j : j \in [T]\}$ be a set of T parties. Without loss of generality, we assume $\mathcal{P}' = \{P_1, \cdots, P_t\}$ to be a t-sized subset of $\mathcal{P}$. A quantum-safe distributed PRF* DPRF*, for input space $\mathcal{X} = \{0,1\}^\star$, key space $\mathcal{K} = \mathcal{R}_q^d$ and output space $\mathcal{Y} = \mathcal{R}_p$, consists of three PPT algorithms,*

$$\mathsf{DPRF} = (\mathsf{DPRF.Setup}, \mathsf{DPRF.PartialEval}, \mathsf{DPRF.FinalEval}).$$

$\underline{\mathsf{DPRF.Setup}(1^\lambda, t, T)}$*: First, the key $\mathbf{K} \xleftarrow{\$} \mathcal{K}$ is sampled. Then it is shared among T parties of $\mathcal{P}$ using linear integer secret sharing of [19]. While applying the secret sharing algorithm, we consider the secret $\mathbf{K}$ to be a vector of $d \cdot N$ number of elements of $\mathbb{Z}_q$. At the end of the secret sharing procedure, each party $P_j \in \mathcal{P}$ holds a total of $\binom{T-1}{t-1}$ shares, each corresponding to one of the t-sized subsets (of $\mathcal{P}$) that P_j can belong to. Each share is also a vector of $d \cdot N$ elements of $\mathbb{Z}_q$. Once the sharing is done, $\mathbf{K}$ is destroyed.*

$\underline{\mathsf{DPRF.PartialEval}(x, P_j, \mathcal{P}')}$*. Assume $\mathbf{K}_j$ is the secret share of P_j corresponding to a t-sized subset $\mathcal{P}'$ such that $P_j \in \mathcal{P}'$. Given an input x, partial evaluation on x by P_j corresponding to $\mathcal{P}'$ is computed using $\mathbf{K}_j$ as $\mu_j = f_{\mathbf{K}_j}(x) = \lfloor \mathcal{H}(x) \cdot \mathbf{K}_j \rceil_{q_1}$.*

$\underline{\mathsf{DPRF.FinalEval}(\{\mu_j\}_{P_j \in \mathcal{P}'}, \mathcal{P}')}$*. With all the t partial evaluations for the parties in $\mathcal{P}'$ available, the final evaluation of distributed PRF on input x is computed as: $f_{\{\mathbf{K}_j\}_{P_j \in \mathcal{P}'}}^{\mathsf{dist}}(x) = \lfloor \mu_1 - \sum_{j=2}^{t} \mu_j \rceil_p$.*

Correctness of the DPRF. Now, we argue the correctness of the proposed MLWR-based DPRF construction in the following.

Theorem 2. *The proposed DPRF in Construction 2, with proper choice of parameters, satisfies correctness and consistency as described in Section 2.4.*

Proof. We require $f^{\mathsf{base}}_{\mathbf{K}}(x)$ and $f^{\mathsf{dist}}_{\{\mathbf{K}_j\}_{P_j\in\mathcal{P}'}}(x)$ to be the same for any choice of $\mathcal{P}'$ to argue the correctness of the proposed DPRF. Using following two facts,

- for any $A \in \mathcal{R}_{N,q}$, $\lfloor A \rceil_p$ can be expressed as $A \cdot \frac{p}{q} + E$, where E is an N-sized error polynomial with each coeffcient in $[\frac{1}{2}, \frac{1}{2})$, and,
- for any minimal valid subset $\mathcal{P}'$ of size t with key shares $\{\mathbf{K}_j\}_{j\in[t]}$, $\mathbf{K} = \mathbf{K}_1 - \sum_{i=2}^{t} \mathbf{K}_j$ holds,

one can follow simple derivation steps to reach the following expressions:

$$f^{\mathsf{base}}_{\mathbf{K}}(x) = \frac{p}{q} \cdot \big(\mathcal{H}(x)\cdot\mathbf{K}\big) + E',\ f^{\mathsf{dist}}_{\{\mathbf{K}_j\}_{P_j\in\mathcal{P}'}}(x) = \frac{p}{q} \cdot \big(\mathcal{H}(x)\cdot\mathbf{K}\big) + \frac{p}{q_1} \cdot \big(E_1 - \sum_{j=2}^{t} E_j\big) + \hat{E},$$

where $E', \{E_j\}_{j\in[t]}, \hat{E}$ are distinct error polynomials with coefficients in $[\frac{1}{2}, \frac{1}{2})$. Clearly,

$$\big|f^{\mathsf{dist}}_{\{\mathbf{K}_j\}_{P_j\in\mathcal{P}'}}(x) - f^{\mathsf{base}}_{\mathbf{K}}(x)\big| \le \frac{p}{q_1} \cdot \big|\sum_{j=1}^{t} E_j\big| + \big|\hat{E} - E'\big|.$$

Note that each coefficient of $\big|\hat{E} - E'\big|$ is less than 1 and each coefficient of $\frac{p}{q_1} \cdot \big|\sum_{j=1}^{t} E_j\big|$ is less than $\frac{pt}{2q_1}$. Hence, with super-polynomial $\frac{q_1}{p}$ ratio, DPRF is correct with overwhelming probability.

However, with practical choice of q_1, p, with polynomial $\frac{q_1}{p}$, we obtain small correctness error, denoted with ϵ_{corr}. While for many applications, a small (but non-negligible) correctness error may be acceptable, for applications demanding higher accuracy, we achieve negligible correctness error by repeating the evaluation parallelly $\mathsf{rep} = \lceil \frac{\lambda}{-\log \epsilon_{\mathsf{corr}}} \rceil$ times for rep different sets of secret shares[1] and accepting the output only if all these outputs match. If all values do not match, we re-evaluate the DPRF in the same manner using some other valid subset. □

Size of secret shares. In Construction 2, each party holds $\binom{T-1}{t-1}$ shares, each belonging to $\mathcal{R}_q^d$, thus requiring $\big(\binom{T-1}{t-1} \cdot N \cdot d \cdot \log q\big)$ bits of storage. When we consider repetition factor rep for negligible correctness error, the storage requirement is increased by a constant multiplication factor of rep, which is $\big(\binom{T-1}{t-1} \cdot N \cdot d \cdot \log q \cdot \mathsf{rep}\big)$ bits. In summary, the asymptotic storage complexity is $O(\binom{T-1}{t-1})$.

3.3 Adaptive Security of MLWR-based DPRF

We first prove the adaptive security of the LWR-based DPRF of [35] in random oracle model (ROM). Then we prove that the construction continues to remain adaptively secure, even when the adversary is allowed quantum access to the random oracle in quantum random oracle model (QROM). Finally, we show how to extend the adaptive security proof to the module setting, which in turn proves the adaptive security of Construction 2 in QROM.

[1] This technique applies to [35] also to achieve negligible correctness error.

3.3.1 Proof of Adaptive Security of LWR-based DPRF [35]

We formalise the adaptive security of the LWR-based DPRF of [35] in ROM in the following theorem.

Theorem 3. *The construction of distributed PRF of [35], described in Section 3.1, is adaptively secure in the random oracle model under LWR assumption.*

Proof. First, we prove the adaptive security of the DPRF in [35], as described in Section 3.1 by showing indistinguishability through a chain of hybrids in ROM. Each hybrid has an initial setup phase and then describes a set of interactions between a challenger Ch and an adversary Ad. The interaction can be broadly categorized into (i) the query phase and (ii) the challenge phase. The query phase (consisting of random oracle queries, partial evaluation queries, and corruption queries in any order) may occur before and after the challenge phase. There can be polynomially many random oracle queries and partial evaluation queries. Challenge phase consists of a single query from Ad with an input of its choice (called the challenge input), and a single response from the Ch which can either be the DPRF evaluation on challenge input or a uniform random value from the DPRF range. Ad is allowed to choose the challenge input arbitrarily, (i.e., the challenge input can be any one of the pre-challenge query inputs), however Ch does not know beforehand which query will correspond to challenge query. Let $Q = \text{poly}(\lambda)$ be the total number of partial evaluation queries sent by Ad to Ch. At the beginning of the game, the challenger guesses a value $c \xleftarrow{\$} [Q+1]$, and considers c^{th} query input to be the challenge input. The game aborts if the guess is wrong. The random guess is correct with non-negligible probability $\frac{1}{Q+1}$, thus, incurring a polynomial loss in the reduction. Also, when $c = Q+1$, it implies that there is no post-challenge partial evaluation query.

Recall that a negligible correctness error is achieved by repeating the complete DPRF evaluation rep times independently for rep different sets of shares for the same base PRF key $\mathbf{k}$, and accepting the DPRF output only if it is the same for all rep cases. Next, we describe the hybrids in detail.

Hybrid_0. This is the real game between Ch and Ad. In the challenge phase, Ad gets to see the result of distributed PRF evaluation on the challenge input.

Setup: Let $\mathcal{P} = \{P_i : i \in [T]\}$ be the set of T parties. Ch chooses a uniformly random PRF secret $\mathbf{k} \xleftarrow{\$} \mathbb{Z}_q^n$, and distributes it among T parties using a (t, T)-threshold secret sharing scheme [19], rep-many times. For each of rep independent sets of shares, each party $P_i \in \mathcal{P}$ receives $\binom{T-1}{t-1}$ secret shares, corresponding to the t-sized subsets of $\mathcal{P}$ that include P_i. Ch maintains a list of compromised secret shares $\mathcal{L}$, which is initially empty. When Ad raises a corruption query asking for the share of P_j with respect to the subset $\mathcal{P}'$ (such that $P_j \in \mathcal{P}'$ and $|\mathcal{P}'| = t$), $\mathcal{L}$ is appended with the entry $(\mathcal{P}', P_j)$. For any fixed subset $\mathcal{P}'$, $\mathcal{L}$ must contain fewer than t entries corresponding to different P_j's. If $\mathcal{L}$ accumulates t or more entries with the same $\mathcal{P}'$ but different P_j's, the secret $\mathbf{k}$ can be reconstructed, allowing the adversary to win the game trivially. Hence, Ch aborts the game on violation of this condition.

Random oracle query: When a query with an input $\mathbf{x}_i$ is received for the first time, the oracle samples a uniform random element from $\mathbb{Z}_q^n$, denoted as $\mathcal{H}(\mathbf{x}_i) \xleftarrow{\$} \mathbb{Z}_q^n$, and stores the pair $(\mathbf{x}_i, \mathcal{H}(\mathbf{x}_i))$, following the lazy sampling strategy in ROM.

Partial evaluation query: Ad submits a query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ asking for the partial evaluation on input $\mathbf{x}_i$ for party P_j and t-sized subset $\mathcal{P}'$, such that $P_j \in \mathcal{P}'$ and $(\mathcal{P}', P_j) \notin \mathcal{L}$.

Let $\mathbf{k}_{j,\mathsf{r}} \forall \mathsf{r} \in [\mathsf{rep}]$ be the secret shares of P_j corresponding to $\mathcal{P}'$, for rep sets of shares. Ch responds with

$$\left[\lfloor\langle\mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,1}\rangle\rceil_{q_1}, \cdots, \lfloor\langle\mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,\mathsf{r}}\rangle\rceil_{q_1}, \cdots, \lfloor\langle\mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,\mathsf{rep}}\rangle\rceil_{q_1}\right] \in \mathbb{Z}_{q_1}^{\mathsf{rep}},$$

an array of rep partial evaluations, each corresponding to one of the rep sets of shares.

Corruption query: Ad raises query of the form $(\mathcal{P}', P_j)$ to Ch such that $P_j \in \mathcal{P}'$, $|\mathcal{P}'| = t$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$. Ch responds with $[\mathbf{k}_{j,1}, \cdots, \mathbf{k}_{j,\mathsf{r}}, \cdots, \mathbf{k}_{j,\mathsf{rep}}] \in \mathbb{Z}_q^{n \times \mathsf{rep}}$, i.e., the secret shares of $(\mathcal{P}', P_j)$ corresponding to rep different sets of shares. The corruption list gets updated as $\mathcal{L} = \mathcal{L} \bigcup \{(\mathcal{P}', P_j)\}$.

Challenge phase: Ad submits a challenge input $\mathbf{x}^\star$ to Ch. Now, Ch computes rep DPRF evaluations on $\mathbf{x}^\star$ as $f^{\mathsf{dist}}_{\{\mathbf{k}_{j,\mathsf{r}}\}_{P_j \in \mathcal{P}'}}(\mathbf{x}^\star)$ for all $\mathsf{r} \in [\mathsf{rep}]$ and checks if all of them are equal. If they are, Ch returns this result to Ad. By the correctness of the DPRF, this value equals $\lfloor\langle\mathcal{H}(\mathbf{x}^\star), \mathbf{k}\rangle\rceil_p$. Here, $\mathcal{P}'$ can be any t-sized subset of $\mathcal{P}$. If any one of the rep evaluations does not match with others, Ch uses some other valid subset to have rep independent evaluations and checks their equality. And, with overwhelming probability the rep evaluations turn out to be equal this time.

Hybrid_1. In this hybrid, Ch responds differently only while answering random oracle queries. For other queries like the partial evaluation query and the corruption query, its behavior remains exactly the same as in Hybrid_0. In the challenge phase also, Ch responds in the same manner as Hybrid_0.

Setup: Two matrices $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and $\mathbf{R} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$ are chosen such that $\mathbf{R}$ is invertible modulo q. If $\mathbf{R}$ is not invertible, then it goes on sampling new $\mathbf{R}$ until it is invertible [1].

Random oracle query: Random oracle is programmed as follows: for every query from Ad with a new input $\mathbf{x}_i$, a short-norm vector $\mathbf{s}_i \in \{0,1\}^m$ is sampled and $\mathcal{H}(\mathbf{x}_i) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i \in \mathbb{Z}_q^{n \times m}$ is returned. For each queried input $\mathbf{x}_i$, $(\mathbf{s}_i, \mathcal{H}(\mathbf{x}_i))$ are stored, following the lazy sampling strategy.

Lemma 1. Ad *can distinguish* Hybrid_1 *from* Hybrid_0 *with advantage* $\epsilon_{0,1}$ *for some* $\epsilon_{0,1} = \mathrm{negl}(\lambda)$.

Proof. Since the two hybrids differ only in the random oracle query phase, we prove that they can not be distinguished from each other from Ch's responses to random oracle queries. Note that for fixed and invertible matrix $\mathbf{R} \in \mathbb{Z}_q^{n \times n}$, and uniform random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{R} \cdot \mathbf{A}$ is also uniform random over $\mathbb{Z}_q^{n \times m}$. Now, the responses to random oracle queries in Hybrid_1, i.e., $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ for any input $\mathbf{x}_i$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$, due to leftover hash lemma [32] for sufficiently large $m \geq 3n \log_2 q$. Thus, the random oracle query phases of Hybrid_0 and Hybrid_1 are statistically indistinguishable. We conclude that Ad has distinguishing advantage of $\epsilon_{0,1}$ for some $\epsilon_{0,1} = \mathrm{negl}(\lambda)$, while transitioning from Hybrid_0 to Hybrid_1. □

Hybrid_2. In this hybrid, only the random oracle query phase differs slightly from Hybrid_1. Interactions between Ch and Ad in all other phases are same as Hybrid_1.

Random oracle query: For any non-challenge input $\mathbf{x}_i$, the response $\mathcal{H}(\mathbf{x}_i)$ is still $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ for some short-norm vector $\mathbf{s}_i$. However, the oracle is programmed differently at the challenge input $\mathbf{x}^\star$ - a short-norm vector $\mathbf{s}^\star \in \{0,1\}^m$ is sampled and $\mathcal{H}(\mathbf{x}^\star) = \mathbf{A} \cdot \mathbf{s}^\star \in \mathbb{Z}_q^n$ is returned.

[1] $\mathbf{R}$ is invertible if it is full-rank and its determinant is co-prime to q. The first event has an approximate probability of 0.28. The second event is independent of the first one and has a probability of 0.5 since we choose q to be a power of 2. Thus $\mathbf{R}$ is invertible with an approximate non-negligible probability of 0.14.

Lemma 2. Ad *can distinguish* Hybrid_2 *from* Hybrid_1 *with an advantage of* $\epsilon_{1,2}$ *for some* $\epsilon_{1,2} = \mathrm{negl}(\lambda)$.

Proof. Since the value of $\mathcal{H}(\mathbf{x}^\star)$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ in both the hybrids, due to leftover hash lemma [32] for sufficiently large $m \geq 3n \log_2 q$, we conclude that Ad has distinguishing advantage of $\epsilon_{1,2}$ for some $\epsilon_{1,2} = \mathrm{negl}(\lambda)$, while transitioning from Hybrid_1 to Hybrid_2. □

$\mathsf{Hybrid}_{3,1}$. In this hybrid, the random oracle query phase is same as Hybrid_2. But the responses to partial evaluation queries and corruption queries are partially simulated - i.e., the responses corresponding to the first set of secret shares are simulated without using the real shares $\mathbf{k}_{j,1}$, but for all other sets where $\mathsf{r} \in [\mathsf{rep}] \setminus \{1\}$, Ch's responses are the same as Hybrid_2.

Partial evaluation query: Broadly, Ch's response to a partial evaluation query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ is in $\mathbb{Z}_{q_1}^{\mathsf{rep}}$, where the first element in the vector is simulated, and the last $(\mathsf{rep}-1)$ are real. For all $\mathsf{r} \in [2, \mathsf{rep}]$, Ch responds with $\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,\mathsf{r}} \rangle \rceil_{q_1}$ in case of any non-challenge input $\mathbf{x}_i$, and with $\lfloor \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}_{j,\mathsf{r}} \rangle \rceil_{q_1}$ in case of challenge input.

However, for $\mathsf{r} = 1$, simulated responses vary for challenge and non-challenge inputs. Notably, Ch does not use the real shares $\mathbf{k}_{j,1}$ for any $(\mathcal{P}', P_j)$.

- *Non-challenge input:* On first query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ for the pair $(\mathcal{P}', P_j)$ such that $P_j \in \mathcal{P}'$, $|\mathcal{P}'| = t$, and $(\mathcal{P}', P_j) \notin \mathcal{L}$, Ch samples a uniform random $\mathbf{k}'_{j,1} \xleftarrow{\$} \mathbb{Z}_q^n$, and responds to Ad with simulated partial evaluation $\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rceil_{q_1}$. Ch continues to use the same $\mathbf{k}'_{j,1}$ to answer queries with other inputs $\mathbf{x}_{i'}$'s for the same pair $(\mathcal{P}', P_j)$ and $\mathsf{r} = 1$.
- *Challenge input:* If the partial evaluation query corresponding to $(\mathcal{P}', P_j)$ is raised for the first time and $\mathcal{L}$ contains less than $(t-1)$ distinct entries for $\mathcal{P}'$, a uniform random $\mathbf{k}'_{j,1} \xleftarrow{\$} \mathbb{Z}_q^n$ is sampled for the pair. Then a simulated share is computed as $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$. The partial evaluation queries on challenge input $\mathbf{x}^\star$ are responded with $\lfloor \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$. Otherwise, if $\mathbf{k}'_{j,1}$ already existed due to previous queries for the pair $(\mathcal{P}', P_j)$, $\mathbf{k}''_{j,1}$ is computed in the same manner, and $\lfloor \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}''_{j,1} \rangle \rceil_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}^\star, \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$ is returned to Ad. Further, if the partial evaluation query corresponding to $(\mathcal{P}', P_j)$ is raised for the first time but $\mathcal{L}$ contains exactly $(t-1)$ distinct entries for $\mathcal{P}'$ (i.e., Ad has reached the corruption threshold), then the response must be consistent with final DPRF evaluation on $\mathbf{x}^\star$ which is visible to Ad in the challenge phase. Hence, the partial evaluation on $\mathbf{x}^\star$ in this case is simulated as $\lfloor v \cdot \frac{q}{q_1} - \sum_{(\mathcal{P}', P_j) \in \mathcal{L}} \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$, where v is the response in the challenge phase. We provide the exact formulation of v in the description of challenge phase.

Note that when Ad is yet to achieve maximal corruption, then, for $\mathsf{r} = 1$, the responses to partial evaluation queries use a random secret $\mathbf{k}'_{j,1}$ while simulating the partial evaluations of non-challenge inputs, but involve use of a simulated secret $\mathbf{k}''_{j,1}$ while responding to partial evaluation queries on challenge input. Informally, responses to partial evaluation queries on challenge input remain *consistent* in the view of the adversary with respect to simulated share $\mathbf{k}''_{j,1}$ which might get revealed eventually (or, have been revealed already) through corruption query phase. Responses to partial evaluation queries on non-challenge inputs do not require special handling when Ad reaches the corruption threshold, because actual DPRF evaluation on non-challenge inputs are never visible to Ad.

Corruption query: For $\mathsf{r} \in [2, \mathsf{rep}]$, Ch responds to corruption query of the form $(\mathcal{P}', P_j)$ with real shares $\mathbf{k}_{j,\mathsf{r}}$, similar to Hybrid_2. However for $\mathsf{r} = 1$, on corruption query of the form $(\mathcal{P}', P_j)$, the

simulated share $\mathbf{k}''_{j,1}$ is computed as $\mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$, and returned to Ad. At this point, it is important to check that the response to corruption query is consistent with the previous responses to partial evaluation queries.

For the non-challenge inputs with $\mathsf{r} = 1$, in the view of the adversary, $\mathcal{H}(\mathbf{x}_i) = (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$, and the compromised share corresponding to the pair $(\mathcal{P}', P_j)$ is $\mathbf{k}''_{j,1}$. So, Ad checks if $\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$ matches with the previous partial evaluation query response $\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rceil_{q_1}$. The check passes since,

$$\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}''_j \rangle \rceil_{q_1} = \lfloor \langle (\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i, \mathbf{k}''_j \rangle \rceil_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{R} \cdot \mathbf{k}''_j \rangle \rceil_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_j \rangle \rceil_{q_1}.$$

For challenge inputs, the consistency is trivially satisfied.

Challenge phase: Upon receiving challenge input $\mathbf{x}^\star$, Ch computes the real DPRF evaluation on $\mathbf{x}^\star$ as $f^{\mathsf{dist}}_{\{\mathbf{k}_{j,r}\}_{P_j \in \mathcal{P}'}}(\mathbf{x}^\star)$ for all $\mathsf{r} \in [2, \mathsf{rep}]$ and checks if all of them are equal. If they are equal, this value is referred to as v (used in answering partial evaluation query on $\mathbf{x}^\star$ in case of maximum corruption), and returned to Ad. Note that Ch's answer to Ad is necessarily the same as in earlier hybrids.

Lemma 3. Ad *can distinguish* $\mathsf{Hybrid}_{3,1}$ *from* Hybrid_2 *with advantage* $\epsilon_{2,3.1}$ *for some* $\epsilon_{2,3.1} = \mathrm{negl}(\lambda)$.

Proof. We prove the phase-wise indistinguishability between $\mathsf{Hybrid}_{3,1}$ and Hybrid_2 in the view of Ad.

- *Partial evaluation query.* For $\mathsf{r} \in [2, \mathsf{rep}]$, the responses are identical to Hybrid_2. So, the focus is on indistinguishability of hybrids in case of $\mathsf{r} = 1$. We discuss three different scenarios in the following:
  - Non-challenge input: Since the responses are of the form $\lfloor \langle \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}'_{j,1} \rangle \rceil_{q_1}$, where $\mathbf{A} \cdot \mathbf{s}_i$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ due to leftover hash lemma and $\mathbf{k}'_{j,1}$ is sampled uniformly at random from $\mathbb{Z}_q^n$, they belong to the same distribution as of real partial evaluations which are of the form $\lfloor \langle \mathcal{H}(\mathbf{x}_i), \mathbf{k}_{j,1} \rangle \rceil_{q_1} = \lfloor \langle \mathbf{R} \cdot \mathbf{A} \cdot \mathbf{s}_i, \mathbf{k}_{j,1} \rangle \rceil_{q_1}$ in Hybrid_2.
  - Challenge input (when corruption threshold is not reached): Here the responses are of the form $\lfloor \langle \mathbf{A} \cdot \mathbf{s}^\star, \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$. Since $\mathbf{A} \cdot \mathbf{s}^\star$ is statistically indistinguishable from uniform random over $\mathbb{Z}_q^n$ due to leftover hash lemma and $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$ is uniform random over $\mathbb{Z}_q^n$ due to uniform randomness of both $\mathbf{R}$ and $\mathbf{k}'_{j,1}$, the simulated partial evaluation essentially comes from the same distribution as of real partial distribution which is of the form $\lfloor \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}_{j,1} \rangle \rceil_{q_1} = \lfloor \langle \mathbf{A} \cdot \mathbf{s}^\star, \mathbf{k}_{j,1} \rangle \rceil_{q_1}$ for uniform random $\mathbf{k}_{j,1}$.
  - Challenge input (when corruption threshold is reached): The simulated partial evaluation of the form $\lfloor v \cdot \frac{q}{q_1} - \sum_{(\mathcal{P}', P_j) \in \mathcal{L}} \langle \mathbf{A} \cdot \mathbf{s}^\star, \mathbf{k}''_{j,1} \rangle \rceil_{q_1}$ is indistinguishable from real partial evaluations $\lfloor \langle \mathcal{H}(\mathbf{x}^\star), \mathbf{k}_{j,1} \rangle \rceil_{q_1}$ by Lemma 1 of [35].
- *Corruption query:* For $\mathsf{r} \in [2, \mathsf{rep}]$, the responses to corruption queries are identical to that in Hybrid_2. For $\mathsf{r} = 1$, the responses are simulated without using real shares as $\mathbf{k}''_{j,1} = \mathbf{R}^{-1} \cdot \mathbf{k}'_{j,1}$, where both $\mathbf{R}$ and $\mathbf{k}'_{j,1}$ are uniform random over $\mathbb{Z}_q^n$. Thus, they are uniform random over $\mathbb{Z}_q^n$, just similar to that of the real shares $\mathbf{k}_{j,1}$ in Hybrid_2.
- *Challenge phase:* In Hybrid_2, Ch computes the DPRF evaluation using real shares of some t-sized subset $\mathcal{P}'$ for each of rep different sets of shares, and responds with the result only

if it is same in all rep cases. In $\mathsf{Hybrid}_{3,1}$, Ch computes the DPRF evaluation using real shares of some t-sized subset $\mathcal{P}'$ for $(\mathsf{rep}-1)$ different sets of shares with $\mathsf{r} \neq 1$, and then behaves identically. It is implied that for $\mathsf{r}=1$, we assume the DPRF evaluation to be same with the real DPRF evaluations, since we simulate the t^{th} partial evaluation on $\mathbf{x}^\star$ using this value (v), in case of maximal corruption. Thus, Ch's response to challenge query is identical in both the hybrids.

Together, we conclude that $\mathsf{Hybrid}_{3,1}$ is statistically indistinguishable from Hybrid_2, and hence Ad has negligible distinguishing advantage, denoted by $\epsilon_{2,3.1}$, while transitioning from Hybrid_2 to $\mathsf{Hybrid}_{3,1}$. □

Next, we have a chain of hybrids represented as $\mathsf{Hybrid}_{3,\mathsf{r}}$ for $\mathsf{r} \in [2, \mathsf{rep}]$. Broadly, $\mathbf{Hybrid_{3,r+1}}$ differs from $\mathbf{Hybrid_{3,r}}$ for all $\mathsf{r} \in [1, \mathsf{rep}-1]$ in the following sense:

- *Partial evaluation query.* For each query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$, Ch provides Ad with rep partial evaluations in both the hybrids, each corresponding to one of rep different sets of shares. In both the hybrids, the first r evaluations are simulated without using real shares, and the last $(\mathsf{rep}-\mathsf{r}-1)$ evaluations are computed using real shares. Thus, the two hybrids differ only in how Ch responds for $(\mathsf{r}+1)^{th}$ evaluation. In $\mathsf{Hybrid}_{3,\mathsf{r}}$ it is real, whereas in $\mathsf{Hybrid}_{3,\mathsf{r}+1}$ it is simulated using the same method as described in $\mathsf{Hybrid}_{3,1}$.
- *Corruption query.* Analogously, in $\mathsf{Hybrid}_{3,\mathsf{r}}$, Ch sends real share $\mathbf{k}_{j,\mathsf{r}+1}$ corresponding to $(\mathsf{r}+1)^{th}$ set of shares whereas in $\mathsf{Hybrid}_{3,\mathsf{r}+1}$, Ch responds with simulated share $\mathbf{k}''_{j,\mathsf{r}+1}$.
- *Challenge phase.* The DPRF evaluation on challenge input $\mathbf{x}^\star$ is decided based on $(\mathsf{rep}-\mathsf{r})$ real evaluations in $\mathsf{Hybrid}_{3,\mathsf{r}}$, whereas in $\mathsf{Hybrid}_{3,\mathsf{r}+1}$ it is based on $(\mathsf{rep}-\mathsf{r}-1)$ real evaluations.

Now, we describe $\mathsf{Hybrid}_{3,\mathsf{rep}}$ in the following, where Ch does not make use of any of the real shares while responding to Ad's queries in all the phases, including the challenge phase. This hybrid can be thought of lying completely in the simulated world.

$\mathbf{Hybrid_{3,rep}}$. Here, the random oracle is programmed in exactly the same manner as in Hybrid_2 (and remain unchanged through $\mathsf{Hybrid}_{3,\mathsf{r}}$ for all $\mathsf{r} \in [\mathsf{rep}]$). We describe all other interactions in the following.

- *Partial evaluation query.* Ch responds to a partial evaluation query of the form $(\mathbf{x}_i, P_j, \mathcal{P}')$ as,
$$\left[\lfloor\langle \mathbf{A}\cdot\mathbf{s}_i, \mathbf{k}'_{j,1}\rangle\rceil_{q_1}, \cdots, \lfloor\langle \mathbf{A}\cdot\mathbf{s}_i, \mathbf{k}'_{j,\mathsf{r}}\rangle\rceil_{q_1}, \cdots, \lfloor\langle \mathbf{A}\cdot\mathbf{s}_i, \mathbf{k}'_{j,\mathsf{rep}}\rangle\rceil_{q_1}\right] \in \mathbb{Z}_{q_1}^{\mathsf{rep}},$$
such that each $\mathbf{k}'_{j,\mathsf{r}} \xleftarrow{\$} \mathbb{Z}_q^n$. In case of challenge input $\mathbf{x}^\star$, if the corruption threshold is not reached by Ad, the response is,
$$\left[\lfloor\langle \mathbf{A}\cdot\mathbf{s}^\star, \mathbf{k}''_{j,1}\rangle\rceil_{q_1}, \cdots, \lfloor\langle \mathbf{A}\cdot\mathbf{s}^\star, \mathbf{k}''_{j,\mathsf{r}}\rangle\rceil_{q_1}, \cdots, \lfloor\langle \mathbf{A}\cdot\mathbf{s}^\star, \mathbf{k}''_{j,\mathsf{rep}}\rangle\rceil_{q_1}\right] \in \mathbb{Z}_{q_1}^{\mathsf{rep}},$$
such that $\mathbf{k}''_{j,\mathsf{r}} = \mathbf{R}^{-1}\cdot\mathbf{k}'_{j,\mathsf{r}}$ holds for already sampled $\mathbf{k}'_{j,\mathsf{r}}$. However, when the maximal corruption threshold is reached for some subset $\mathcal{P}'$, the t^{th} honest partial evaluation on $\mathbf{x}^\star$ is simulated as $\mu_{\mathsf{sim},\mathsf{r}} = \lfloor v_{\mathsf{sim}}\cdot\frac{q}{q_1} - \sum_{(\mathcal{P}',P_j)\in\mathcal{L}}\langle \mathbf{A}\cdot\mathbf{s}^\star, \mathbf{k}''_{j,\mathsf{r}}\rangle\rceil_{q_1}$, where v_{sim} is the response of Ch in the challenge phase. Thus, the response becomes,
$$\left[\mu_{\mathsf{sim},1}, \cdots, \mu_{\mathsf{sim},\mathsf{r}}, \cdots, \mu_{\mathsf{sim},\mathsf{rep}}\right] \in \mathbb{Z}_{q_1}^{\mathsf{rep}}.$$

- *Corruption query.* In response to any corruption query of the form $(\mathcal{P}', P_j)$, Ch returns all rep simulated shares, i.e., $\left[\mathbf{k}''_{j,1}, \cdots, \mathbf{k}''_{j,\mathsf{r}}, \cdots, \mathbf{k}''_{j,\mathsf{rep}}\right] \in \mathbb{Z}_q^{n \times \mathsf{rep}}$.

- *Challenge phase.* Ch samples $v_{\mathsf{sim}} \xleftarrow{\$} \mathbb{Z}_p$ and sends it as response to challenge query $\mathbf{x}^\star$.

Lemma 4. Ad *distinguishes* $\mathsf{Hybrid}_{3,\mathsf{rep}}$ *from* Hybrid_2 *with advantage* $(\mathsf{rep}-1)\cdot\epsilon_{2,3.1}+\epsilon'$*, where* $\epsilon' = \mathrm{negl}(\lambda)$.

Proof. Since the transition from $\mathsf{Hybrid}_{3,\mathsf{r}}$ to $\mathsf{Hybrid}_{3,\mathsf{r}+1}$ for all $\mathsf{r} \in [\mathsf{rep}-1]$ is exactly similar to the transition from Hybrid_2 to $\mathsf{Hybrid}_{3,1}$, and each transition introduces a distinguishing advantage of $\epsilon_{2,3.1}$ for Ad, it gives a total distinguishing advantage of $(\mathsf{rep}-1)\cdot\epsilon_{2,3.1}$ between Hybrid_2 and $\mathsf{Hybrid}_{3,\mathsf{rep}-1}$ for Ad.

The transition from $\mathsf{Hybrid}_{3,\mathsf{rep}-1}$ to $\mathsf{Hybrid}_{3,\mathsf{rep}}$ is similar to that of the transition between any $\mathsf{Hybrid}_{3,\mathsf{r}}$ and $\mathsf{Hybrid}_{3,\mathsf{r}+1}$, for $\mathsf{r} \in [\mathsf{rep}-1]$ with respect to the partial evaluation query phase and the corruption query phase. However, in challenge phase the transition is different. Ch replaces the real DPRF evaluation with a uniform random value in $\mathbb{Z}_p$. Note that by the correctness of DPRF, the real DVRF evaluation in $\mathsf{Hybrid}_{3,\mathsf{rep}-1}$ is equal to that of base PRF evaluation which is $f_{\mathbf{k}}^{\mathsf{base}}(\mathcal{H}(\mathbf{x}^\star)) = \lfloor\langle\mathcal{H}(\mathbf{x}^\star), \mathbf{k}\rangle\rceil_p$, with $\mathbf{k}$ being the master secret key. And by the LWR assumption, this value is computationally indistinguishable from a uniform random value v_{sim} in $\mathsf{Hybrid}_{3,\mathsf{rep}}$. We assume Ad's distinguishing advantage to be ϵ' for some $\epsilon' = \mathrm{negl}(\lambda)$ while transitioning from $\mathsf{Hybrid}_{3,\mathsf{rep}-1}$ to $\mathsf{Hybrid}_{3,\mathsf{rep}}$. □

Together, we conclude that Ad can distinguish between Hybrid_2 and $\mathsf{Hybrid}_{3,\mathsf{rep}}$ with advantage $(\mathsf{rep}-1)\cdot\epsilon_{2,3.1}+\epsilon'$.

Next, we have $(\mathsf{rep}-1)$ hybrids represented as $\mathsf{Hybrid}_{4,\mathsf{r}}$ in decreasing order of r for $\mathsf{r} \in [\mathsf{rep}-1]$. Each $\mathsf{Hybrid}_{4,\mathsf{r}}$ is identical to $\mathsf{Hybrid}_{3,\mathsf{r}}$ for all $\mathsf{r} \in [\mathsf{rep}-1]$, except in challenge phase. In $\mathsf{Hybrid}_{3,\mathsf{r}}$, Ch responds with real DPRF evaluation whereas in $\mathsf{Hybrid}_{4,\mathsf{r}}$, Ch responds with a uniform random value in $\mathbb{Z}_p$. The indistinguishability while transitioning from $\mathsf{Hybrid}_{4,\mathsf{r}+1}$ to $\mathsf{Hybrid}_{4,\mathsf{r}}$ can be argued in the similar manner as we do in case of transitioning from $\mathsf{Hybrid}_{3,\mathsf{r}}$ to $\mathsf{Hybrid}_{3,\mathsf{r}+1}$. The responses to partial evaluation queries and corruption queries are transitioned from simulated to real for one of the rep sets of shares, one at a time for each hybrid transition.

To summarize, Ad can distinguish $\mathbf{Hybrid_{4,1}}$ from $\mathbf{Hybrid_{3,rep}}$ with advantage $(\mathsf{rep}-1)\cdot\epsilon_{2,3.1}$.

Next, for the sake of completeness, we have three more hybrids $\mathbf{Hybrid_5}$, $\mathbf{Hybrid_6}$, $\mathbf{Hybrid_7}$, which are identical to Hybrid_2, Hybrid_1 and Hybrid_0, respectively, except the fact that in the challenge phase, Ch responds with uniform random values in $\mathbb{Z}_p$ in the former cases and with real DPRF evaluations in the later.

Note that the interactions between Ch and Ad in the final hybrid Hybrid_7 lie in the real world, but the response in the challenge phase is a uniform random value. Ad's views in the first and last hybrid (i.e., Hybrid_0 and Hybrid_7) emulate the notion of pseudorandomness of a DPRF under adaptive corruption, as given in Section 2.4. Since for a specific choice of parameters, $\mathsf{rep} = O(1)$ is a constant, Ad can distinguish Hybrid_7 from Hybrid_0 with advantage,

$$\frac{1}{Q+1}\Big(2\epsilon_{0,1} + 2\epsilon_{1,2} + (2\cdot\mathsf{rep}-1)\cdot\epsilon_{2,3.1} + \epsilon'\Big) = \mathrm{negl}(\lambda).$$

The multiplicative factor of $\frac{1}{Q+1}$ is there because Ch's guess of the challenge input $\mathbf{x}^\star$ is correct with probability $\frac{1}{Q+1}$. Thus, we conclude that the LWR-based distributed PRF in [35] is adaptively secure in the random oracle model (ROM). □

Next, we argue the adaptive security of LWR-based DPRF of [35] in QROM.

Theorem 4. *The construction of LWR-based distributed PRF in [35] is adaptively secure in quantum random oracle model (QROM).*

Proof. Though the adaptive proof of security of LWR-based DPRF in [35] in ROM can not immediately be imported to QROM settings, where Ad can query the random oracle (RO) in superposition, we prove that with slight modification in the proof it is possible to argue adaptive security of the DPRF even in QROM.

Before proceeding further, we emphasize that the concept of lazy sampling in ROM does not work in QROM. This is because Ad can query RO in superposition of exponential number of inputs with varying amplitudes. So, the RO (i.e., the function $\mathcal{H}(\cdot)$) must be (conceptually) initialized with the complete input-output behavior in the beginning of the game. Note that only this conceptual shift suffices to prove the *static* security of LWR-based DPRF in QROM in [35][1].

However that is not the case for the proof of adaptive security, because the proof heavily relies on the programmability of the RO. Again, unlike classical ROM, programming the RO output at the unqueried inputs is not applicable in QROM, because there is no concept of "unqueried input" in QROM due to superpositioned queries, and hence this manipulation can be detected by Ad. Note that though there are a total of $(2 \cdot \mathsf{rep} + 5)$ hybrids in the proof of adaptive security in ROM, the RO description is modified only during transition from Hybrid_0 to Hybrid_1, Hybrid_1 to Hybrid_2 (and analogously during transition from Hybrid_5 to Hybrid_6, Hybrid_6 to Hybrid_7). Since RO's response remains identical throughout all the other hybrids, Ad does not have any extra advantage to distinguish between them by exploiting its quantum access to RO. So, we focus on two following transitions:

- $\mathsf{Hybrid}_0 \rightarrow \mathsf{Hybrid}_1$: Since, $(\mathbf{R} \cdot \mathbf{A}) \cdot \mathbf{s}_i$ is statistically close to $\mathcal{H}(\mathbf{x}_i)$, these two hybrids are indistinguishable in the view of a quantum polynomial time (QPT) adversary.
- $\mathsf{Hybrid}_1 \rightarrow \mathsf{Hybrid}_2$: Considering that the RO is initialized in the beginning of the game in both hybrids, note that the RO in these two hybrids differ only at a single input, namely $\mathbf{x}^\star$. Distinguishing between these two instantiations of RO is equivalent to the problem of searching the differing input position (which is $\mathbf{x}^\star$) using polynomial number of superpositioned RO queries. Ad does not know $\mathbf{x}^\star$ at the time of RO initialization; because Ch guesses the challenge input beforehand and aborts the game later if the guess is wrong. Then by [7], a QPT adversary has a distinguishing advantage of $O(\frac{Q'^2}{q^n})$, where $Q' = \mathrm{poly}(\lambda)$ is the total number of RO queries.

Thus, we conclude that LWR-based DPRF in [35] is adaptively secure in QROM. □

3.3.2 Adaptive Security of MLWR-based DPRF

The following theorem formalises the adaptive security of our proposed MLWR-based DPRF.

Theorem 5. *Construction 2 is adaptively secure in the quantum random oracle model if MLWR assumption holds.*

[1] In particular, in the proof of static security of [35] no RO programming is involved and hence the RO's response to Ad's queries remain necessarily same throughout all the four hybrids, thus providing no extra advantage to Ad in distinguishing between those hybrids, even with quantum access to the RO.

Proof. The proof approach closely follows the adaptive security proof, previously outlined for the LWR-based DPRF, except with a few differences.

An alternative of leftover hash lemma in module settings: We use the leftover hash lemma [32] as an important tool to simulate the RO in $\mathsf{Hybrid}_{\mathsf{sim}}$. Analogously, to program the random oracle in module setting, we need a mechanism to output a uniform random element of $\mathcal{R}_q^d$. We resort to the following regularity lemma from [26].

Theorem 6 (Adapted from Theorem 4.1 and Corollary 4.2 of [26])**.** *Let* $\mathbf{A} = [\mathbf{I}_d|\tilde{\mathbf{A}}] \leftarrow \mathcal{R}_q^{d\times\ell}$*, such that* $\tilde{\mathbf{A}} \xleftarrow{\$} \mathcal{R}_q^{d\times(\ell-d)}$ *is uniformly random and* $\mathbf{I}_d$ *is the identity matrix. Then with probability* $1 - 2^{-\Omega(n)}$ *over the choice of* $\tilde{\mathbf{A}}$*, the distribution of* $\mathbf{A}\cdot\mathbf{s} \in \mathcal{R}_q^d$*, where each element of* $\mathbf{s} \in \mathcal{R}_q^\ell$ *is sampled from a discrete Gaussian distribution of radius* $r > 2n \cdot q^{\frac{d}{\ell}+\frac{2}{n\ell}}$ *over* $\mathcal{R}$*, is within statistical distance* $2^{-\Omega(n)}$ *of the uniform distribution over* $\mathcal{R}_q^d$*.*

The theorem implies that, $\mathbf{A}\cdot\mathbf{s} \in \mathcal{R}_q^d$ can be used to program the output of random oracle if $\mathbf{A}$ and $\mathbf{s}$ come from the above-mentioned distributions. Also, we replace the matrix $\mathbf{R}$ of LWR setting with a scalar $r \xleftarrow{\$} \mathcal{R}_q$ in MLWR setting such that r is invertible modulo $(x^N + 1)$. Note that with $\mathbf{A} = [\mathbf{I}_d|\tilde{\mathbf{A}}] \in \mathcal{R}_q^{d\times\ell}$, $r\cdot\mathbf{A}$ performs scalar multiplication on each element of $\mathbf{A}$ with r, and the output is $[\mathbf{I}'_d|r\cdot\tilde{\mathbf{A}}]$. Here, $\mathbf{I}'_d$ is a diagonal matrix with each diagonal element initialized to r. From [26], we conclude that $r\cdot\mathbf{A}$ preserves the matrix format that is eligible for regularity lemma to be applied, i.e., $(r\cdot\mathbf{A})\cdot\mathbf{s}$ is uniform random over $\mathcal{R}_q^d$ for suitable distribution of $\mathbf{s}$. The rest of the proof essentially proceeds in a similar manner.

Concluding the proof. Replacing the leftover hash lemma with the regularity lemma from [26] and a few obvious changes in dimensions and parameters translate the proof of adaptive security of LWR-based DPRF into a proof for Construction 2. □

3.4 Choice of Parameters for Adaptive DPRF

We summarize the concrete choice of parameters for both the adaptively secure DPRFs. Notably, in the partial evaluation phase of these adaptive DPRFs, the rounding step transitions values from modulus q to modulus q_1; hence, the error here is of the order of $\frac{q}{q_1}$. In other words, the modulus-to-noise ratio is of the order $\frac{q}{e} = \frac{q}{\frac{q}{q_1}} = q_1$. The solution to average-case LWR problem with a smaller modulus-to-noise ratio implies a solution to worst-case lattice problem with a smaller approximation factor. Hence, the smaller we keep the modulus-to-noise ratio, the harder it is to solve the underlying worst-case lattice problem. While choosing concrete parameters for practical instantiations, our objective is to ensure that the modulus-to-noise ratio $\frac{q}{e}$ remains sufficiently small to mitigate any known attack strategies. We further elaborate on the concrete choice of parameters for both the instantiations of adaptive DPRFs in the following.

LWR-based Adaptive DPRF. The following parameter set for the LWR-based DPRF, as suggested by [35], ensures 128-bit security in static corruption scenarios: $n = 512$, $q = 2^{32}$, $q_1 = 2^{28}$, $p = 2^{10}$. By proof of Theorem 3, the exact same parameter set should also provide adaptive security. However, in this work, we use a different parameter set, namely, $n = 1024, q = 2^{64}$, $q_1 = 2^{38}$, $p = 2^{10}$, while providing experimental results for adaptively secure LWR-based DPRF. Intuitively, the modulus q, the error proportional to $\frac{q}{q_1}$, and the dimension n together correspond to the security of DPRF whereas the ratio $\frac{q_1}{p}$ corresponds to the correctness of DPRF. Our concrete choice of parameters provides at least 128 bits of security considering the

attacks listed in *lattice estimator*[1] [4].

MLWR-based Adaptive DPRF. We use the following parameter set for MLWR-based DPRF: $N = 256$, $d = 4$, $q = 2^{64}$, $q_1 = 2^{38}$, $p = 2^{10}$ to provide at least 128 bits of security. The reasons behind choosing this parameter set for our construction are the following: (i) there are currently no known attack on MLWE with these parameters, which are actually comparable to those used in Kyber [11] (ii) there are no meaningful attack on MLWR that are better than those on MLWE (in the same way as there are no meaningful attack on LWR that are better than LWE attacks), and (iii) there are no meaningful attack on RLWE or MLWE that are better than LWE attacks [3].

4 EQuADiSE: An Application

We consider Distributed Symmetric-key Encryption (DiSE) as a potential application of distributed PRFs. DiSE was first introduced by [2], where DPRF served as a fundamental building block, and the security of DiSE was implied by the security of the DPRF. Following the same line of work, [29] constructed adaptive DiSE by incorporating an adaptively secure version of DDH-based DPRF. More recently, [35] constructed PQDiSE, where the underlying DPRF is instantiated with an LWR-based DPRF, and post-quantum security of the DPRF implies post-quantum security of PQDiSE. In this work, we introduce EQuADiSE (Efficient Quantum-safe Adaptive DiSE), wherein the DPRF has two different instantiations: one is LWR-based adaptive DPRF and another is MLWR-based adaptive DPRF (Section 3.2). Our experiment in Section 5 shows that EQuADiSE, with both the instantiations of adaptive DPRF, performs better than PQDiSE (which is not adaptively secure) and adaptive DiSE (which is not quantum-safe). We further observe that EQuADiSE, when instantiated with MLWR-based adaptive DPRF, outperforms the LWR-based adaptive DPRF instantiation since MLWR-based DPRF is more *compact* than LWR-based DPRF.

Now, for the sake of completeness, we briefly discuss the original work of DiSE [2]. We then mention several instantiations of DPRFs from the literature with their detailed constructions in Appendix A, and their performance will be compared in Section 5.

4.1 DiSE: An Overview [2]

Distributed Symmetric-key Encryption (DiSE) enables the distributed computation of both encryption and decryption in a symmetric encryption scheme, i.e., encryption and decryption are performed by multiple servers in a distributed manner. Specifically in t-out-of-T DiSE, whenever a plaintext (or ciphertext) needs to be encrypted (or decrypted), a service request is initiated, and any t encryption (or decryption) servers out of T are contacted to process the request. Each selected server performs a partial computation using its individual key share and contributes to the formation of a message-specific symmetric key in a distributed fashion. This symmetric key is then used as the encryption (or decryption) key for the actual plaintext (or ciphertext). The formal construction of DiSE is provided below.

Construction 3 (Distributed Symmetric-key Encryption (DiSE), adapted from [2])**.** *Let $\mathcal{P} = \{P_j : j \in [T]\}$ be the set of encryption/decryption servers. DiSE internally uses the following cryptographic primitives as its building blocks:*

[1] https://github.com/malb/lattice-estimator/commit/a51a410192e39d7df133c775bbe058ecacb5577d

(i) A DPRF $\mathsf{DPRF} = (\mathsf{DPRF.Setup}, \mathsf{DPRF.PartialEval}, \mathsf{DPRF.FinalEval})$,
(ii) A PRG (pseudo random generator) of polynomial stretch,
(iii) A commitment scheme $\mathsf{C} = (\mathsf{C.Setup}, \mathsf{C.Com})$.

On top the above primitives, DiSE is a collection of three PPT algorithms,

$$\mathsf{DiSE} = (\mathsf{DiSE.Setup}, \mathsf{DiSE.DistEncrypt}, \mathsf{DiSE.DistDecrypt}).$$

DiSE.Setup*:* $\mathsf{DPRF.Setup}(1^\lambda, t, T)$ *provides DPRF key shares* k_j *to* P_j $\forall j \in [T]$ *and* $\mathsf{C.Setup}(1^\lambda)$ *outputs public parameters for the commitment scheme.*

DiSE.DistEncrypt*: An encryptor* E *encrypts a plaintext* m *as follows. It first computes a commitment* α *on the message* m *using randomness* ρ*, and then sends it to* t *encryption servers. Each server performs partial evaluation on* α *using its secret share* k_j *and computes* μ_j*. On receiving all* t μ_j*'s,* E *combines them to generate symmetric key* $\tilde{k}$*, which is used to encrypt* m*. The ciphertext has the form* $c = (\alpha, e)$*, where* $e = \mathsf{PRG}(\tilde{k}) \oplus (m||\rho)$.

DiSE.DistDecrypt*: A decryptor* D*, when provided with a ciphertext* $c = (\alpha, e)$*, contacts a set of* t *decryption servers and provides them with* α*. Each server computes partial evaluation* μ_j *on* α*, and sends it to* D*.* D *combines them all to generate the symmetric key* $\tilde{k}$*. Now* $(m||\rho)$ *is retrieved from* e*, by XORing it with* $\mathsf{PRG}(\tilde{k})$*. Plaintext* m *is obtained by discarding the randomness* ρ.

4.2 Instantiations of DPRF

We now discuss several instances of Distributed PRF. DiSE [2] utilizes two different DPRF constructions. One is based on the Decisional Diffie-Hellman (DDH) assumption, as introduced in [30], while the other is derived from AES-128. The AES-128-based DPRF leverages the combinatorial technique from [30], which transforms any PRF into a DPRF, with AES-128 functioning as the underlying PRF. The AES-based DPRF also serves as a generic MPC-based technique for evaluating AES. These two DPRFs are described in Box 1 and 2 of Appendix A. The AES-based combinatorial construction offers adaptive security only when $\binom{T}{t}$ is polynomial. To handle arbitrary t, T, the adaptive DiSE construction in [29] introduces an adaptive DDH-based DPRF, which replaces the original DDH-based DPRF in DiSE [2], as detailed in Box 3 of Appendix A. For post-quantum security, PQDiSE [35] employs an LWR-based DPRF construction, described in Section 3.1. A quantum-resilient adaptive DPRF based on the Learning With Errors (LWE) assumption from [24] is outlined in Box 4 of Appendix A. Additionally, any PRF can be distributed using Threshold Fully Homomorphic Encryption (ThFHE) as *Universal Thresholdizer* [10]. Box 5 in Appendix A provides further details on this DPRF construction, using AES as the underlying PRF. Table 1 in the introduction already offers an asymptotic comparison of these DPRFs, and we validate this comparison through experiments in Section 5. As the DPRF module is the sole component responsible for distributed computation (encryption/decryption) in DiSE, the asymptotic time complexity of DPRF evaluation directly translates to the time complexity of distributed encryption/decryption in various DiSE variants. Table 2 illustrates this point, along with a detailed comparison of the storage requirements for each party to store their key shares. We exclude LWE-based DPRF and ThFHE-based DPRF from integrating into DiSE due to their significantly high concrete evaluation time.

DiSE variants	Underlying DPRF	Key Share Size (in bits)	Distributed Encryption Overheads	Quantum-safe?	Adaptively secure?
Original DiSE [2]	AES(128)-based	$128 \times \binom{T-1}{t-1}$	$O(\binom{T-1}{t-1})$	No	Yes
Original DiSE [2]	DDH-based	256×1	$O(t)$	No	No
Adaptive DiSE [29]	Adaptive DDH-based	512×1	$O(t)$	No	Yes
PQDiSE [35]	LWR-based	$512 \times 32 \times \binom{T-1}{t-1}$	$O(t)$	Yes	No
EQuADiSE	LWR-based	$1024 \times 64 \times \binom{T-1}{t-1}$	$O(t)$	Yes	Yes
EQuADiSE	MLWR-based	$(256 \times 4) \times 64 \times \binom{T-1}{t-1}$	$O(t)$	Yes	Yes

Table 2: Comparison of DiSE variants implying EQuADiSE to be only efficient DiSE that is both quantum-safe and adaptively secure

5 Experimental Evaluation

In this section, we compare the performance of our proposed post-quantum adaptive LWR-based and MLWR-based DPRFs in terms of evaluation time, relative to several existing DPRFs. These include the AES-based combinatorial DPRF from [2], the DDH-based DPRF from [2], the adaptive DDH-based DPRF from [29], the adaptive quantum-safe LWE-based DPRF from [24], the quantum-safe LWR-based DPRF from [35], and the quantum-safe ThFHE-based DPRF utilizing the *Universal Thresholdizer* (UT) tool [10], all within semi-honest adversarial settings. We implement the ThFHE-based DPRF using the TFHE library [1] [15], with AES as the underlying PRF. For the ThFHE implementation, we adopt the parameter choices recommended by [16]. Figure 1, Figure 2 and Figure 3 depict the comparison across all the DPRFs in terms of partial evaluation time, final evaluation time and total evaluation time respectively. Subsequently, we plug all the DPRFs into the original DiSE [2] construction and evaluate their throughput (number of encryptions per second) in Figure 4 across all DiSE instances. Among them, EQuADiSE, which has LWR-based and MLWR-based adaptive DPRF at its core, turns out to have the highest throughput. All experiments have been executed on a high-end server with an Intel(R) Xeon(R) Silver 4214R CPU (2.40GHz clock frequency), 48 cores, 128GB RAM. The x-axis of all graphs presents the total number of parties T in $(\frac{T}{2}, T)$-DPRFs. The y-axis of all graphs is presented on a logarithmic scale. We use NTL library[2] to support arithmetic over $\mathbb{Z}_q^n$ and $\mathcal{R}_q$. We use Blake2[3] to instantiate the hash function, modelled as a random oracle. Our implementation is available here [4]. In the following, we provide a detailed analysis of the plots presented in Figures 1,2,3,4.

Partial evaluation time vs. (t, T) values [Figure 1]. Since each of the t parties computes the partial evaluation in parallel, we plot the maximum partial evaluation time required by any of the t participating parties. The notably poor performance of ThFHE-based DPRF in partial evaluation stems from the computational overhead of FHE evaluation on the AES circuit, followed by its partial decryption. Partial evaluation of LWE-based DPRF involves multiple matrix multiplications (ref. Box 4), which leads to a higher partial evaluation time. MLWR-based adaptive DPRF, on the other hand, exhibits the lowest partial evaluation time

[1] https://tfhe.github.io/tfhe/

[2] https://libntl.org/

[3] https://www.blake2.net/

[4] https://github.com/SayaniSinha97/EQuADiSE

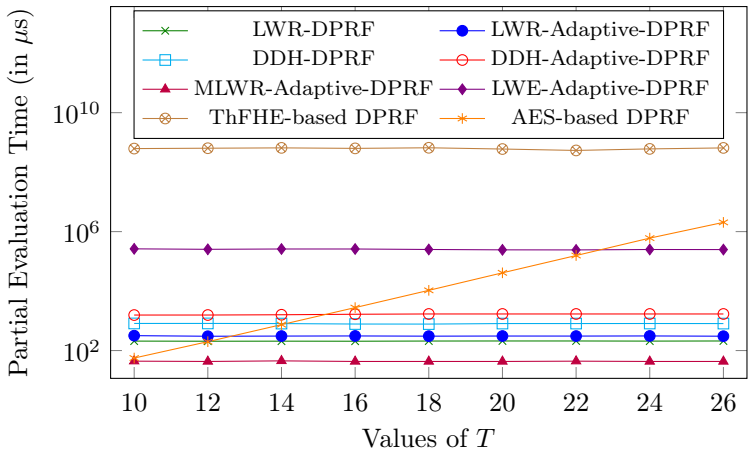

Figure 1: Partial evaluation time of $(\frac{T}{2}, T)$-DPRFs

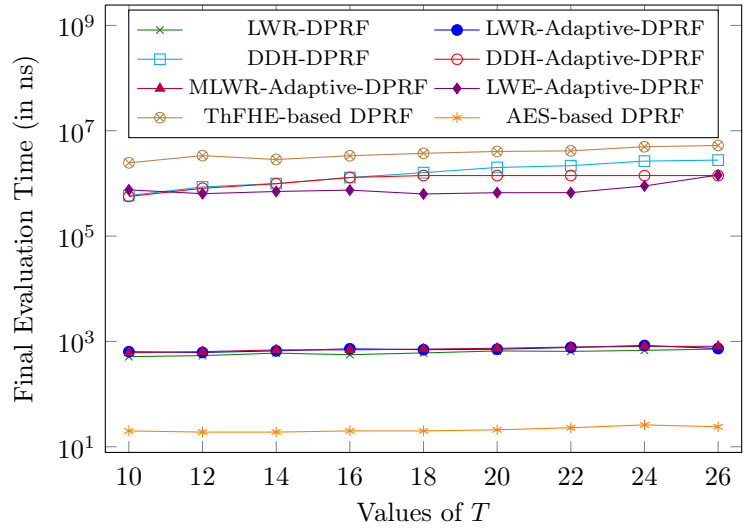

Figure 2: Final evaluation time of $(\frac{T}{2}, T)$-DPRFs

across almost all values of T, outperforming the other DPRF instances. For the AES-based combinatorial DPRF, the partial evaluation time increases exponentially (appearing linear in the log-scale plot) with larger T, as the maximum partial evaluation time is proportional to $\binom{T-1}{t-1}$. This differs from the other DPRFs, where the computational load of partial evaluation phase is evenly distributed among the t participating parties and individual partial evaluation time is independent of t or T. While the proposed LWR-based adaptive DPRF, due to larger parameters, slightly underperforms compared to the non-adaptive LWR-based DPRF, it still demonstrates significantly shorter partial evaluation times than the adaptive DDH-based, LWE-based, and ThFHE-based DPRFs. The adaptive DDH-based DPRF incurs higher partial evaluation times compared to the standard DDH-based DPRF, as it requires two modular exponentiations within a DDH-hard group, rather than one. MLWR-based DPRF has less partial evaluation time than LWR-based adaptive DPRF on average over multiple iterations, since we leverage the fact that one evaluation of MLWR-based DPRF generates a total of (256×10) pseudorandom bits (since

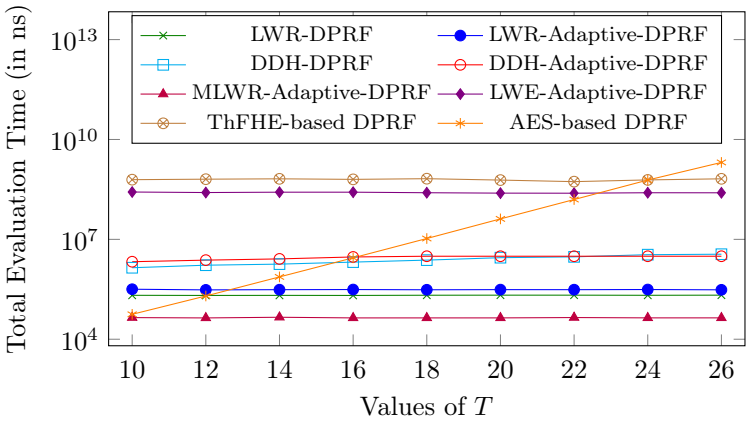

Figure 3: Total evaluation time of $(\frac{T}{2}, T)$-DPRFs

$N = 256$, $p = 2^{10}$). In contrast, a single LWR-based DPRF evaluation generates only 10 bits.

Final evaluation time vs. (t, T) values [Figure 2]. AES-based combinatorial DPRF has the least final evaluation time since it involves only XORing of t partial evaluations. Three plots of LWR-based DPRF, adaptive LWR-based and adaptive MLWR-based DPRF for final evaluation time overlap with each other because the final evaluation consists of similar operations (addition over a ring and round off) in all three cases. All of them perform better than DDH-based, LWE-based and ThFHE-based AES DPRF. Adaptive DDH-based DPRF takes a similar time as its non-adaptive counterpart since it requires the same number of modular exponentiation and modular multiplication. Final evaluation of LWE-based DPRF consists of $O(n \log q)$ rounding-off operations followed by a deterministic randomness extractor function, thus having a higher final evaluation time than LWR-based and MLWR-based DPRF. Final evaluation time of ThFHE-based AES DPRF is the same as the final decryption time of an FHE ciphertext that encrypts the AES output.

Total evaluation time vs. (t, T) values [Figure 3]. Please note that the y-axis in Figure 1 and Figure 2 are in (logarithmic scale) microseconds and nanoseconds, respectively. We plot the total (partial+final) evaluation time (in nanoseconds) for all the DPRFs in this graph. The graph clearly depicts the efficiency of our proposed adaptive MLWR-based DPRF with respect to other DPRFs, considering end-to-end time for DPRF evaluation.

Throughput vs. (t, T) values [Figure 4]. Since LWE-based adaptive DPRF and ThFHE-based DPRF perform worse than the other DPRFs (ref. Figure 3), we exclude them from being plugged into DiSE. This graph plots the throughput of DiSE (number of encryptions performed in one second), when the DPRF module of DiSE is instantiated with the following DPRFs: (i) DDH-based DPRF (original DiSE), (ii) AES-based combinatorial DPRF (original DiSE), (iii) DDH-based adaptive DPRF (adaptive DiSE [29]), (iv) LWR-based DPRF (PQDiSE), (v) LWR-based adaptive DPRF (EQuADiSE), (vi) MLWR-basd adaptive DPRF (EQuADiSE). Clearly, EQuADiSE achieves a significantly better throughput, as expected from its shorter evaluation time (Figure 3). Moreover, MLWR-based EQuADiSE achieves higher throughput than its LWR-based counterpart. This is because a single evaluation of the MLWR-DPRF generates 256×10 pseudorandom bits, enabling the encryption of twenty 128-bit plaintexts in a batch. In contrast, LWR-DPRF requires 13 evaluations to encrypt a single 128-bit plaintext, as each evaluation produces only 10 pseudorandom bits.

Considering the implications from all the graphs above, we conclude that the total evaluation

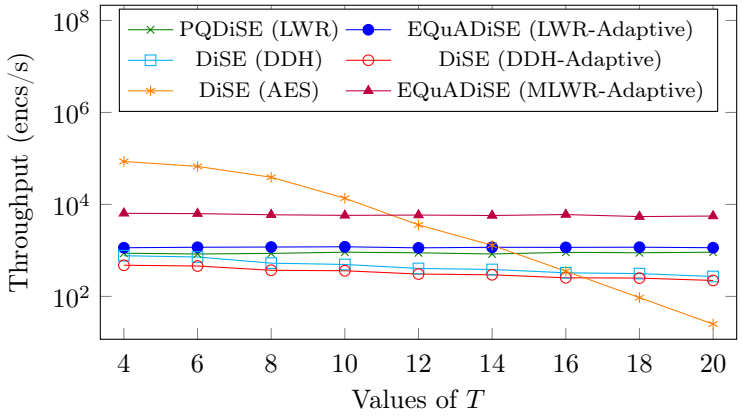

Figure 4: Throughput in DiSE, PQDiSE, EQuADiSE

time of the proposed MLWR-based adaptive DPRF is effectively less than those of existing DPRFs, and its resilience against adaptive corruption makes it suitable quantum-safe candidate for practically efficient real-world symmetric-key cryptosystems. EQuADiSE validates this conclusion by improving upon DiSE both in terms of performance and security.

References

[1] Shashank Agrawal, Wei Dai, Atul Luykx, Pratyay Mukherjee, and Peter Rindal. Paradise: efficient threshold authenticated encryption in fully malicious model. In *International Conference on Cryptology in India*, pages 26–51. Springer, 2022.

[2] Shashank Agrawal, Payman Mohassel, Pratyay Mukherjee, and Peter Rindal. Dise: distributed symmetric-key encryption. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1993–2010, 2018.

[3] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, et al. Homomorphic encryption standard. *Protecting privacy through homomorphic encryption*, pages 31–62, 2021.

[4] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[5] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold fhe. In *Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings 31*, pages 483–501. Springer, 2012.

[6] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 719–737. Springer, 2012.

[7] Charles H Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM journal on Computing*, 26(5):1510–1523, 1997.

[8] Andrej Bogdanov, Siyao Guo, Daniel Masny, Silas Richelson, and Alon Rosen. On the hardness of learning with rounding over small modulus. In *Theory of Cryptography Conference*, pages 209–224. Springer, 2016.

[9] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *International conference on the theory and application of cryptology and information security*, pages 41–69. Springer, 2011.

[10] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter MR Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38*, pages 565–596. Springer, 2018.

[11] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

[12] Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8,*

2023, Proceedings, Part I, volume 14438 of *Lecture Notes in Computer Science*, pages 371–404. Springer, 2023.

[13] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367. Springer, 2015.

[14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[15] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[16] Siddhartha Chowdhury, Sayani Sinha, Animesh Singh, Shubham Mishra, Chandan Chaudhary, Sikhar Patranabis, Pratyay Mukherjee, Ayantika Chatterjee, and Debdeep Mukhopadhyay. Efficient threshold fhe with application to real-time systems. *Cryptology ePrint Archive*, 2022.

[17] Mihai Christodorescu, Sivanarayana Gaddam, Pratyay Mukherjee, and Rohit Sinha. Amortized threshold symmetric-key encryption. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2758–2779, 2021.

[18] Morten Dahl, Daniel Demmler, Sarah El Kazdadi, Arthur Meyre, Jean-Baptiste Orfila, Dragos Rotaru, Nigel P Smart, Samuel Tap, and Michael Walter. Noah's ark: Efficient threshold-fhe using noise flooding. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 35–46, 2023.

[19] Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In *International Workshop on Public Key Cryptography*, pages 75–90. Springer, 2006.

[20] Yvo G Desmedt. Threshold cryptography. *European Transactions on Telecommunications*, 5(4):449–458, 1994.

[21] Pousali Dey, Pratyay Mukherjee, Swagata Sasmal, and Rohit Sinha. Hise: Hierarchical (threshold) symmetric-key encryption. Cryptology ePrint Archive, Paper 2024/158, 2024. `https://eprint.iacr.org/2024/158`.

[22] Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *Progress in Cryptology–AFRICACRYPT 2018: 10th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 7–9, 2018, Proceedings 10*, pages 282–305. Springer, 2018.

[23] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[24] Benoît Libert, Damien Stehlé, and Radu Titiu. Adaptively secure distributed prfs from lwe. *Journal of Cryptology*, 34(3):1–49, 2021.

[25] Benoît Libert and Radu Ţiţiu. Multi-client functional encryption for linear functions in the standard model from lwe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 520–551. Springer, 2019.

[26] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-lwe cryptography. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 35–54. Springer, 2013.

[27] Daniele Micciancio and Adam Suhl. Simulation-secure threshold pke from lwe with polynomial modulus. *Cryptology ePrint Archive*, 2023.

[28] Florian Le Mouël, Maxime Godon, Renaud Brien, Erwan Beurier, Nora Boulahia-Cuppens, and Frédéric Cuppens. Trustless distributed symmetric-key encryption. *arXiv preprint arXiv:2408.16137*, 2024.

[29] Pratyay Mukherjee. Adaptively secure threshold symmetric-key encryption. In *International Conference on Cryptology in India*, pages 465–487. Springer, 2020.

[30] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and kdcs. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 327–346. Springer, 1999.

[31] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. In *38th Annual Symposium on Foundations of Computer Science, FOCS '97, Miami Beach, Florida, USA, October 19-22, 1997*, pages 458–467. IEEE Computer Society, 1997.

[32] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.

[33] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

[34] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.

[35] Sayani Sinha, Sikhar Patranabis, and Debdeep Mukhopadhyay. Efficient quantum-safe distributed prf and applications: Playing dise in a quantum world. In *International Conference on Applied Cryptography and Network Security*, pages 47–78. Springer, 2024.

A Overview of Several Existing DPRFs

In this section, we provide a detailed overview of the following existing DPRFs: (i) Box-1: DDH-based DPRF, as used in DiSE [2], (ii) Box2: Combinatorial AES-based DPRF, as used in DiSE [2], (iii) Box-3: DDH-based Adaptive DPRF, as used in adaptive DiSE [29], (iv) Box-4: LWE-based Adaptive DPRF, as described in [24], (v) Box-5: ThFHE-based DPRF from AES, using *Universal Thresholdizer* (UT) tool of [10].

The DDH-based DPRF [BOX-1]

Let $G = \langle g \rangle$ be a multiplicative cyclic group of prime order p in which the DDH assumption holds and $\mathcal{H} : \{0,1\}^\star \to G$ be a hash function modelled as a random oracle. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t.

- $\mathsf{Setup}(1^\lambda, t, T)$: Sample a secret $s \xleftarrow{\$} \mathbb{Z}_p$ and then distribute it among T parties using Shamir's secret sharing algorithm such that each P_i gets s_i as its share.
- $\mathsf{PartialEval}(x, P_i, \mathcal{P}')$: Party P_i computes partial evaluation $\mu_i = \mathcal{H}(x)^{s_i}$ using its secret share s_i, and sends it to rest of the $(t-1)$ parties of $\mathcal{P}'$.
- $\mathsf{FinalEval}(\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}')$: Once all t partial evaluations of the parties in $\mathcal{P}'$ are together, final DPRF output is computed as $\prod_{P_i \in \mathcal{P}'} \mu_i^{\ell_{i,\mathcal{P}'}}$, where $\ell_{i,\mathcal{P}'}$ corresponds to Lagrange's coefficient of P_i for group $\mathcal{P}'$.

The AES-based DPRF [BOX-2]

Let $f : \{0,1\}^\lambda \times \{0,1\}^\star \to \{0,1\}^\star$ be the AES function. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t.

- $\mathsf{Setup}(1^\lambda, t, T)$: Sample $k_1, \cdots, k_d \xleftarrow{\$} \{0,1\}^\lambda$, where $d = \binom{T}{T-t+1}$. Let $S_1, \cdots, S_d$ be d distinct $(T-t+1)$-sized subsets of $\mathcal{P}$. Each k_i is given to all the $(T-t+1)$ parties in S_i, $\forall i \in [d]$. Effectively, each party holds $\binom{T-1}{T-t}$ number of shares.
- $\mathsf{PartialEval}(x, P_i, \mathcal{P}')$: P_i computes $\mu_i = \{f_{k_j}(x) \mid P_i \text{ owns } k_j\}$.
- $\mathsf{FinalEval}(\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}')$: Sparse each μ_i as $\{\mu_{i,j} \mid P_i \text{ owns } k_j\}$. From all μ_i for $P_i \in \mathcal{P}'$, extract a set of $\mu_{i,j}$ such that exactly one $\mu_{i,j}$ is there for each $j \in [d]$. We 'xor' all such $\mu_{i,j}$ to get the final evaluation.

The Adaptive DDH-based DPRF [BOX-3]

Let $G = \langle g \rangle$ be a multiplicative cyclic group of prime order p in which the DDH assumption holds and $\mathcal{H} : \{0,1\}^\star \to G^2$ be a hash function modelled as a random oracle. Let $\mathcal{P} = \{P_i : i \in [T]\}$ be a set of T parties, and $\mathcal{P}' \subset \mathcal{P}$ denotes a subset of size t.

- $\mathsf{Setup}(1^\lambda, t, T)$: Sample secret $(u, v) \xleftarrow{\$} \mathbb{Z}_p^2$ and then distribute it among T parties by applying Shamir's secret sharing scheme individually on both u and v, such that each P_i gets (u_i, v_i) as its share.
- $\mathsf{PartialEval}(x, P_i, \mathcal{P}')$: Party P_i computes partial evaluation $\mu_i = w_1^{u_i} w_2^{v_i}$, such that $\mathcal{H}(x) = (w_1, w_2) \in \mathbb{Z}^2$, and sends it to rest of the $(t-1)$ parties of $\mathcal{P}'$.
- $\mathsf{FinalEval}(\{\mu_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}')$: Once all t partial evaluations of the parties in $\mathcal{P}'$ are together, final DPRF output is computed as $\prod_{P_i \in \mathcal{P}'} \mu_i^{\ell_{i,\mathcal{P}'}}$, where $\ell_{i,\mathcal{P}'}$ corresponds to Lagrange's coefficient of P_i for group $\mathcal{P}'$.

Quantum-safe Adaptive LWE-based DPRF [BOX-4]

Let q, q_1, p be three moduli with sufficiently large $\frac{q}{q_1}$ and $\frac{q_1}{p}$ ratio. Let $n, m = \text{poly}(\lambda)$, where λ is the security parameter and $m \geq 2n \log q$. Let $AHF : \{0,1\}^{\ell} \to \{0,1\}^{L}$ be a balanced admissible hash function, and $\pi : \mathbb{Z}_{q_1}^m \to \mathbb{Z}_p^k$ be a deterministic randomness extractor. $\mathcal{P} = \{P_i : i \in [T]\}$ is a set of T parties and $\mathcal{P}' \subset \mathcal{P}$ is a subset of size t.

- $\mathsf{Setup}(1^{\lambda}, t, T)$: Choose random matrices $\mathbf{A}_0 \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and $\mathbf{A}_{i,b} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, for each $i \in [L]$ and $b \in \{0,1\}$ such that each $\mathbf{G}^{-1}(\mathbf{A}_{i,b}) \in \mathbb{Z}^{m \times m}$ is $\mathbb{Z}_q$-invertible. The secret $\mathbf{s} \in \mathbb{Z}_q^n$ is a small-norm vector sampled from a Gaussian distribution. Apply linear integer secret sharing (LISSS) [19] to share it among T parties.
- $\mathsf{PartialEval}(x \in \{0,1\}^{\ell}, P_i, \mathcal{P}')$: First, $\tilde{x} = AHF(x) \in \{0,1\}^{L}$ is computed. Next, input-dependent matrix $\mathbf{A}(\tilde{x}) = \mathbf{A}_0 \cdot \prod_{i=1}^{L} \mathbf{G}^{-1}(\mathbf{A}_{i,\tilde{x}_i})$ is computed. Then the partial evaluation, computed by party P_i is $\boldsymbol{\mu}_i = \lfloor \mathbf{A}(\tilde{x})^{\top} \cdot \mathbf{s}_i \rceil_{q_1} \in \mathbb{Z}_q^m$, where $\mathbf{s}_i$ is the secret share of P_i corresponding to $\mathcal{P}'$.
- $\mathsf{FinalEval}(\{\boldsymbol{\mu}_i\}_{P_i \in \mathcal{P}'}, \mathcal{P}')$: Once all t partial evaluations corresponding to t-sized subset $\mathcal{P}'$ is received, they are combined to get the final DPRF output as follows: $\pi(\lfloor \sum_{i=1}^{t} c_i \boldsymbol{\mu}_i \rceil_p)$, where c_i's are the integer coefficients required for reconstructing secret $\mathbf{s}$ form shares of t parties.

ThFHE-based AES DPRF [BOX-5]

- Setup: FHE-encrypt the AES input (i.e., the PRF input). Distribute the FHE-decryption key (i.e., the PRF key) among multiple parties using threshold secret sharing of [19].
- PartialEval: Each party performs homomorphic evaluation of the AES circuit in the presence of encrypted AES input and evaluation key; then performs partial decryption on the encrypted AES output and returns the partial decryption as partial DPRF evaluation.
- FinalEval: With all the parties' partial evaluations combined, final evaluation of DPRF is equivalent to final decryption phase of threshold FHE. Partial descriptions are combined and decoded to retrieve the output of AES evaluation in plaintext.